

Wiederholung

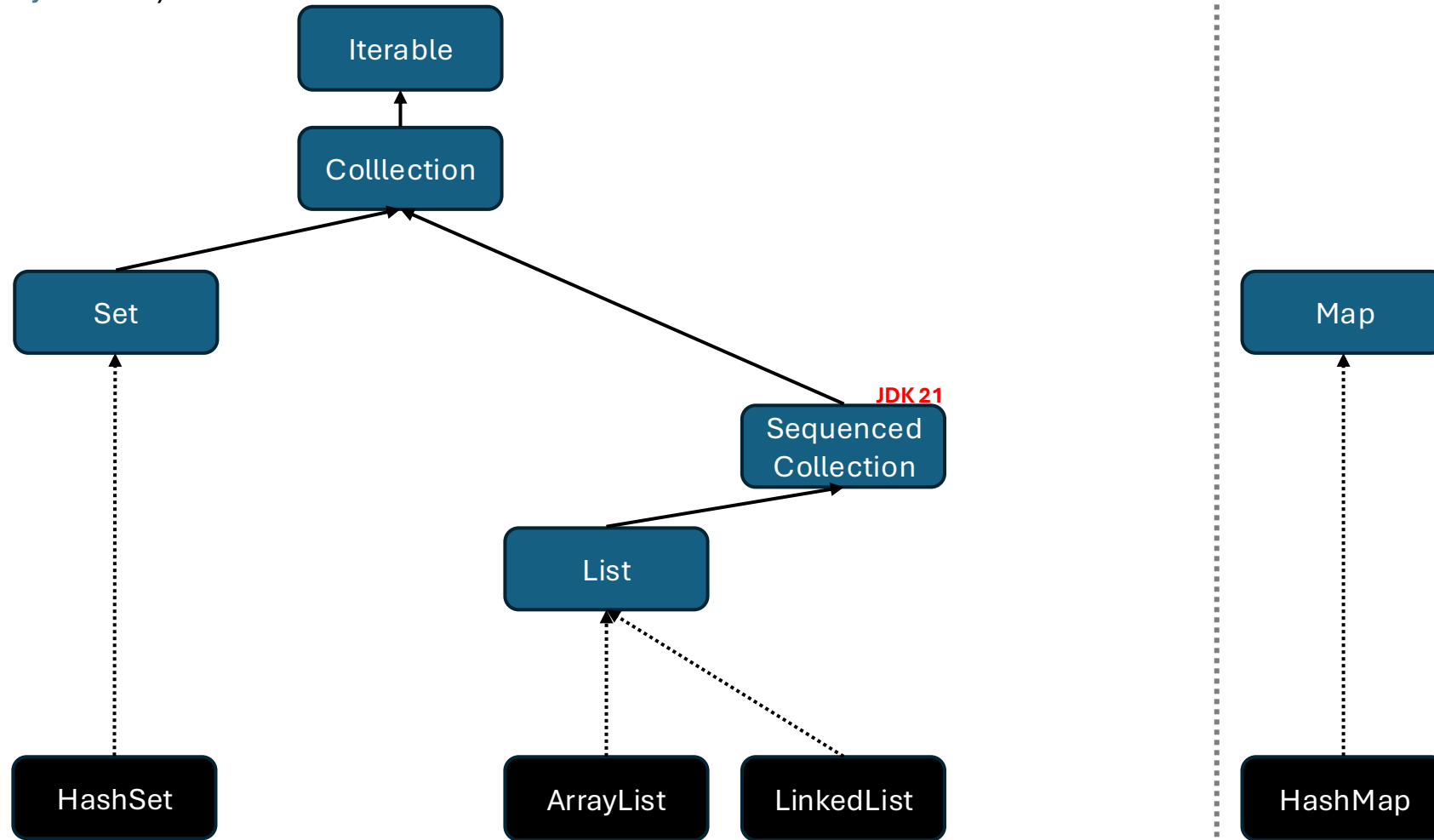


Collections

- Array (Wiederholung)
- Generics <T>
- Iterable
- Collection
- List
- Set
- Map

Java Collections Framework

(ein Ausschnitt aus [java.util](https://docs.oracle.com/javase/8/docs/api/java/util/))



Legende:

Interface

Klasse

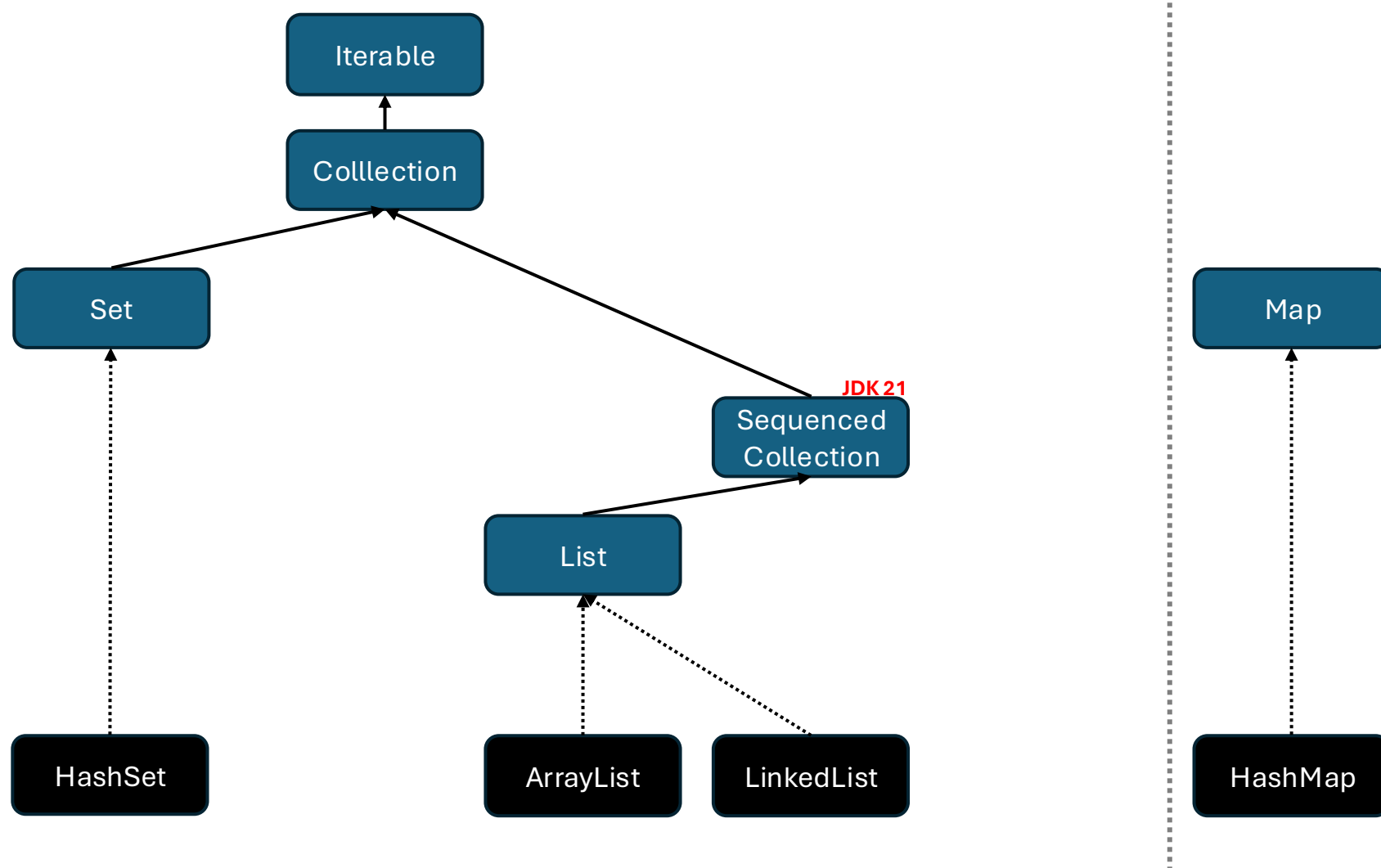
Abstr.

Klasse

extends

implements

Wir schauen uns nur einen Teil an 😊



Iterable

Definition „iterieren“ von Oxford:

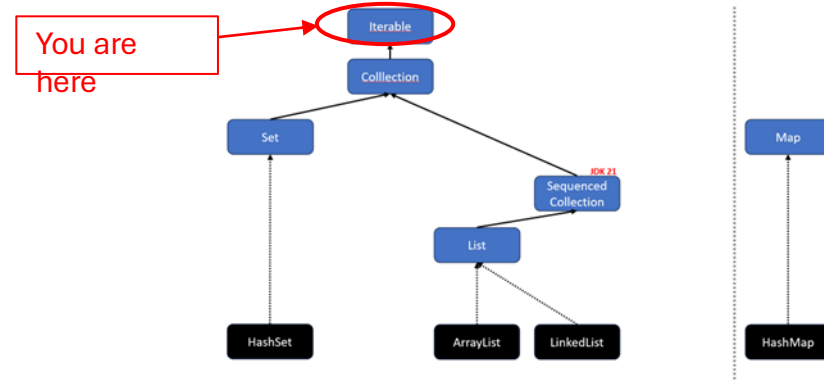
1. (ein Wort, eine Wortgruppe) im Satz wiederholen
2. wiederholen (EDV)

Wenn eine Klasse das Interface Iterable implementiert, kann man ihre Elemente mithilfe eines Iterators durchlaufen. Über die Methode `next()` erhalten wird immer das nächste Element.

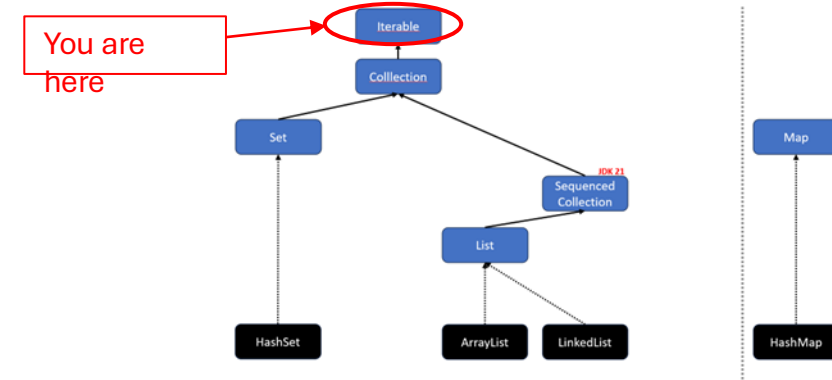
```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {0, 1, 2, 3, 4};  
        Iterator iterator = Arrays.stream(numbers).iterator();  
        while (iterator.hasNext()) {  
            Object current = iterator.next();  
            System.out.print(current + " ");  
        }  
    }  
}
```

Ausgabe:

0 1 2 3 4



Iterable<E>



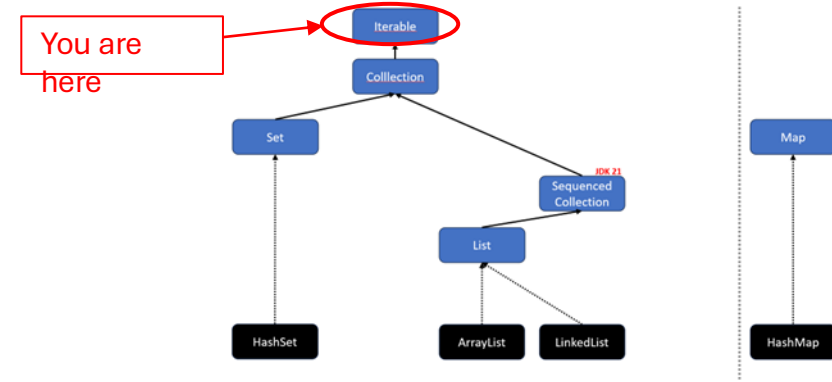
Mithilfe von Generics ist es möglich, ein `Iterable<E>` zu erstellen, das einem bestimmten Datentyp verwendet und zurückgibt.

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {0, 1, 2, 3, 4};  
        Iterator<Integer> iterator = Arrays.stream(numbers).iterator();  
        while (iterator.hasNext()) {  
            int number = iterator.next(); // Java unboxes Integer to int  
            System.out.print(number + " ");  
        }  
    }  
}
```

Ausgabe:

0 1 2 3 4

Iterable: for-each



Das Interface `Iterable` erlaubt es außerdem, mit dem Java-Schlüsselwort *for* über die Elemente zu iterieren 🤖.

🤖 FYI: *for* funktioniert auch mit Arrays.

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {0, 1, 2, 3, 4};  
        for(int number: numbers) {  
            System.out.println(number + " ");  
        }  
    }  
}
```

Ausgabe:

0 1 2 3 4

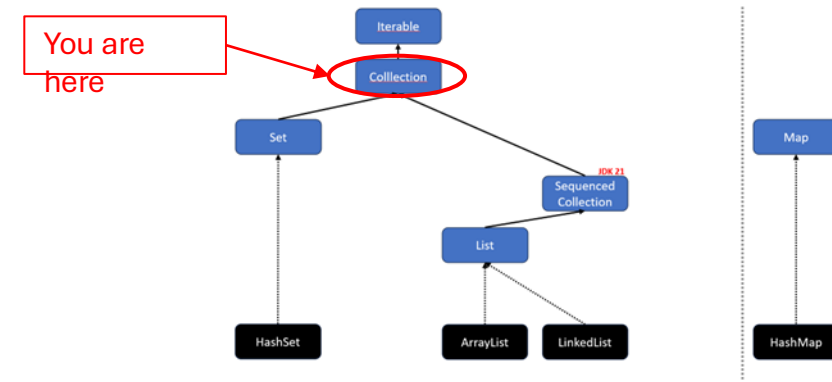
Übung: One-dimensional String-Theory

1. Erstelle ein Array “names” mit folgenden Strings:
"Chani", "Sara", "Leto"
2. Iteriere über das Array mit einem Iterator und gebe jeden Namen in GROßBUCHSTABEN in der Konsole aus.
3. Iteriere nochmal über das Array, dieses Mal mit einer for-Schleife. Ersetze hierbei in jeden Namen das “o” mit einem “i” und gebe den neuen Namen in der Konsole aus.

1. Erstelle ein Array "names" mit folgenden Strings:
"Chani", "Sara", "Leto"
2. Iteriere über das Array mit einem Iterator und gebe jeden Namen in GROßBUCHSTABEN in der Konsole aus.
3. Iteriere nochmal über das Array, dieses Mal mit einer for-Schleife. Ersetze hierbei in jeden Namen das "o" mit einem "i" und gebe den neuen Namen in der Konsole aus.

```
1  import java.util.Arrays;
2  import java.util.Iterator;
3
4  public class Demo {
5      static void main(String[] args) {
6          // 1. Erstelle ein Array "names" mit drei String-Elementen.
7          String[] names = {"Chani", "Sara", "Leto"};
8
9          // Erzeuge einen Iterator, um über das 'names'-Array zu laufen.
10         // Der Iterator ist typsicher und weiß, dass er Strings enthält (<String>).
11         Iterator<String> iterator = Arrays.stream(names).iterator();
12
13         // 2. Iteriere über das Array mit einem Iterator.
14         // Die while-Schleife läuft, solange der Iterator noch ein nächstes Element hat.
15         while (iterator.hasNext()) {
16             // Hole das nächste Element (einen Namen) aus dem Iterator.
17             String name = iterator.next();
18             // Wandle den Namen in Großbuchstaben um und gib ihn in der Konsole aus.
19             System.out.println(name.toUpperCase());
20         }
21
22         // 3. Iteriere nochmal über das Array, dieses Mal mit einer for-each-Schleife.
23         // Diese Schleife ist eine bequemere, besser lesbare Alternative zum Iterator.
24         for (String name : names) {
25             // Ersetze in jedem Namen den Buchstaben "o" durch "i" und gib das Ergebnis aus.
26             System.out.println(name.replace(target: "o", replacement: "i"));
27         }
28     }
29 }
30
```

Collection<E>



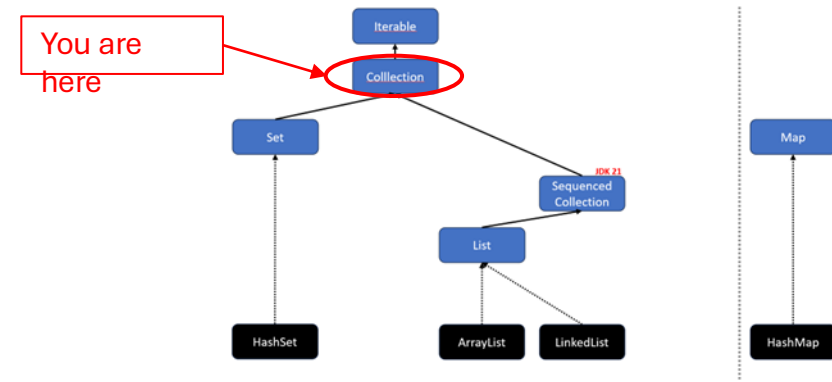
Das Interface Collection<E> bietet grundlegende Methoden, um Elemente zu hinzuzufügen oder zu entfernen.

„A collection represents a group of objects, known as its *elements*. Some collections allow duplicate elements and others do not. Some are ordered and others unordered.”

```
boolean add(E e)
boolean addAll(Collection c)
```

```
boolean remove(Object o)
boolean removeAll(Collection c)
```

Collection<E> – Methoden



Weitere nützliche Methoden des Collection-Interfaces:

// Enthält die Datenstruktur das Element o?

`boolean` `contains(Object o)`

// Gibt die Anzahl der enthaltenen Elemente zurück

`int` `size()`

// Gibt ein Object-Array mit allen Elementen zurück

`Object[]` `toArray()`

// Gibt einen Stream mit der Collection als Quelle zurück

`Stream<E>` `stream()` **SPOILER
ALERT!**

Collection – contains und equals

Collection bietet eine Methode *contains(Object object)*, mit der geprüft werden kann, ob die Collection (z. B. ArrayList) ein bestimmtes Objekt enthält. Für die Prüfung auf Gleichheit wird die *equals()-Methode* für jedes Objekt in der Collection aufgerufen.

Um die Gleichheitsdefinition für Objekte einer Klasse anzupassen, kann *equals()* überschrieben werden:

```
class Project {  
    private String name;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        if (this == object) {  
            return true;  
        }  
        if (object == null || this.getClass() != object.getClass()) {  
            return false;  
        }  
        Project otherProject = (Project) object;  
        return Objects.equals(this.name, otherProject.name);  
    }  
}
```

Collection – contains, equals und hashCode

Wichtig: Wenn equals überschrieben wird, **muss** auch die Methode hashCode (aus der Klasse Object) überschrieben werden! Beide Methoden, equals und hashCode, müssen dieselben Attribute verwenden.

```
class Project {  
    private String name;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        if (this == object) { return true; }  
  
        if (object == null || this.getClass() != object.getClass()) { return false; }  
  
        Project otherProject = (Project) object;  
        return Objects.equals(this.name, otherProject.name);  
    }  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public int hashCode() {  
        return Objects.hashCode(this.name);  
    }  
}
```



Der Hashcode wird berechnet, indem die Eigenschaften des Objekts in eine Ganzzahl umgewandelt werden.

Collection – contains, equals und hashCode

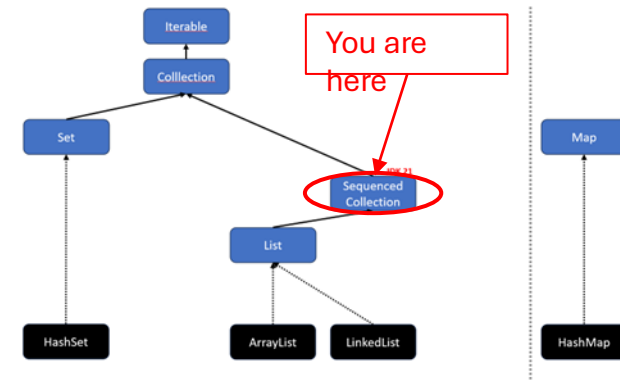
Wichtig: Wenn equals überschrieben wird, **muss** auch die Methode hashCode (aus der Klasse Object) überschrieben werden! Beide Methoden, equals und hashCode, müssen dieselben Attribute verwenden.

Angenommen Project bekommt ein weiteres Attribut "private String owner;". Zwei Projekte sind gleich, wenn sowohl der Name als auch der Owner gleich sind. D. h., dass in equals und in hashCode beide Attribute verwendet werden müssen!

```
class Project {  
    private String name;  
    private String owner;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        if (this == object) { return true; }  
  
        if (object == null || this.getClass() != object.getClass()) { return false; }  
  
        Project otherProject = (Project) object;  
        return Objects.equals(this.name, otherProject.name) &&  
            Objects.equals(this.owner, otherProject.owner);  
    }  
  
    @Override  
    public int hashCode() {  
        return 31 * Objects.hashCode(name) + Objects.hashCode(owner);  
    }  
}
```

SequencedCollection<E>

JDK 21

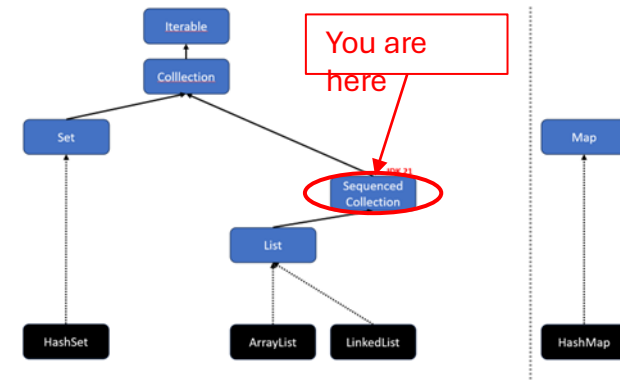


„A collection that has a well-defined encounter order, that supports operations at both ends, and that is reversible. The elements of a sequenced collection have an *encounter order*, where conceptually the elements have a linear arrangement from the first element to the last element. Given any two elements, one element is either before (closer to the first element) or after (closer to the last element) the other element.”

Das Interface [SequencedCollection](#)<E> erweitert das Interface `Collection`<E> und erlaubt es zudem das erste oder letzte Element zu erhalten oder zu ändern.

<code>void addFirst(E e)</code>	<code>E getFirst()</code>
<code>void addLast(E e)</code>	<code>E getLast()</code>
<code>E removeFirst()</code>	<code>SequencedCollection<E> reversed()</code>
<code>E removeLast()</code>	

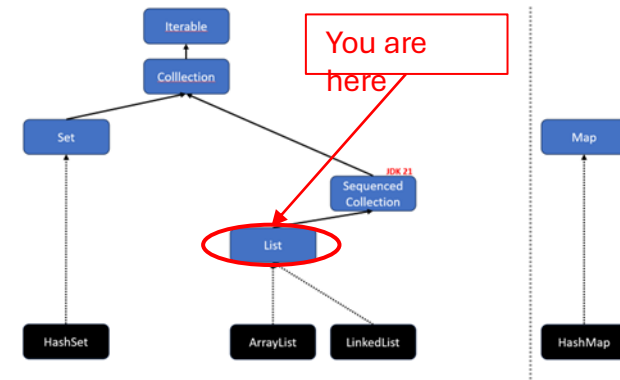
SequencedCollection<E>^{JDK 21}



“Motivation: Java’s Collections Framework lacks a collection type that represents a sequence of elements with a defined encounter order. It also lacks a uniform set of operations that apply across such collections. These gaps have been a repeated source of problems and complaints.”

	Get First Element	Get Last Element
List	<code>list.get(0)</code>	<code>list.get(list.size() - 1)</code>
Deque	<code>deque.getFirst()</code>	<code>deque.getLast()</code>
SortedSet	<code>sortedSet.first()</code>	<code>sortedSet.last()</code>
LinkedHashSet	<code>linkedHashSet.iterator().next()</code>	-

List<E>



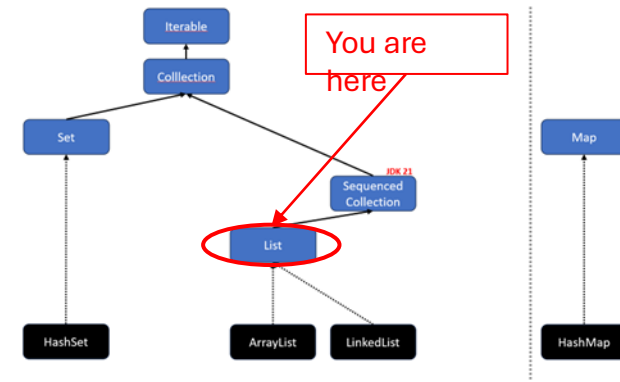
„An ordered collection (also known as a sequence). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list. Unlike sets, lists typically allow duplicate elements.”

Das Interface [List](#)<E> erweitert das Interface `SequencedCollection`<E> und erlaubt zudem auf bestimmte Elemente direkt zuzugreifen z. B. mit `get` (siehe nächste Folie).

Häufig genutzte Implementierungen:

- `ArrayList`
- `LinkedList`

List<E> – Methoden



List<E> erweitert SequencedCollection<E> um weitere Methoden:

// Gibt das Element an der Position index zurück

E get(int index)

// Fügt das Element an Position index zur Liste hinzu

void add(int index, E element)

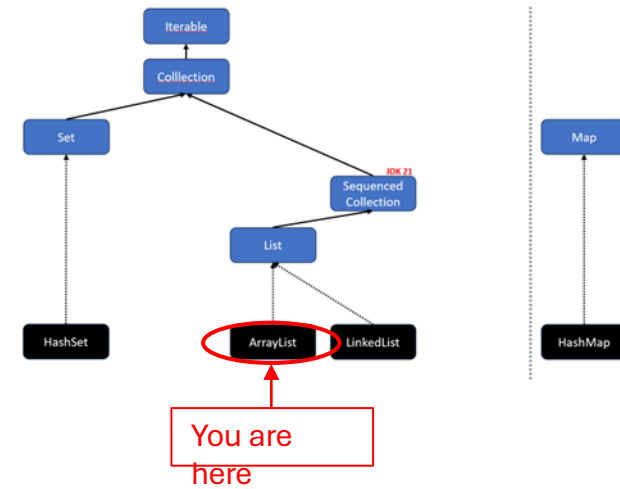
// Ersetzt das Element an der Position index und gibt das alte Element zurück

E set(int index, E element)

// Entfernt das Element an der Position index und gibt das entfernte Element zurück

E remove(int index)

ArrayList<E>



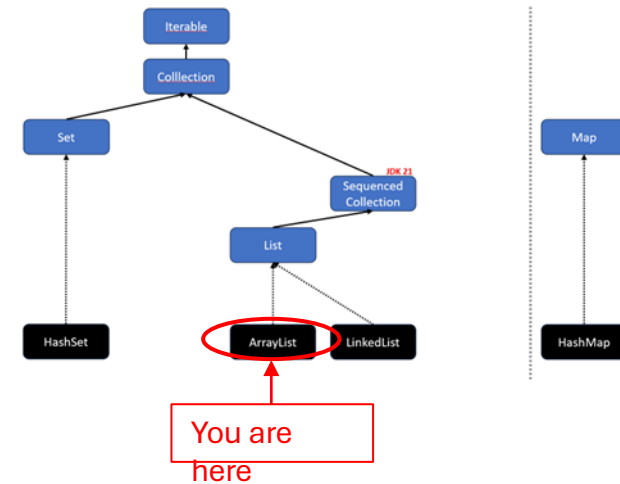
```
public class Main {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Sara");
        names.add("Alia");
        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Ausgabe:

Sara
Alia

🧐 FYI: Alternativ kannst du Folgendes schreiben: `var names = new ArrayList<String>();`
Beachte hierbei, dass der Datentyp `String` auf die rechte Seite gewandert ist, da Java ihn sonst nicht automatisch ermittelt kann.

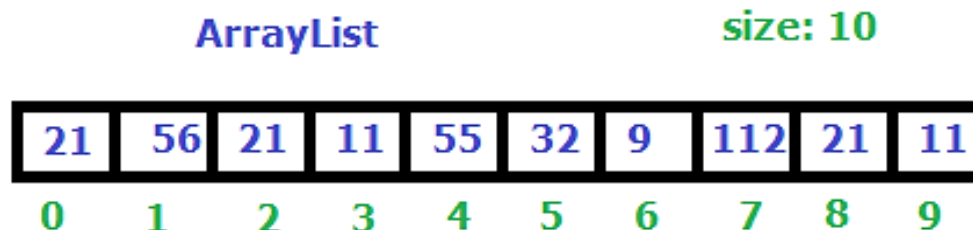
ArrayList<E>



Eine [ArrayList](#)<E> erweitert das Interface `List`<E>. Eine `ArrayList` ist im Grunde wie ein **Array mit variabler Länge** und bietet Methoden, um Elemente zu erhalten oder zu ändern (z. B. `add`, `remove`, `set`, `contains`).

```
var numbers = new ArrayList<Integer>();
```

```
numbers.addAll(List.of(21, 56, 21, 11, 55, 32, 9, 112, 21, 11));
```



Good to know:

Eine `ArrayList` verwendet intern ein gewöhnliches Array, das wenn nötig vergrößert (verdoppelt) wird. Das Vergrößern erfordert allerdings ein kostspieliges Umkopieren aller Elemente in ein größeres Array.

List.of(E...)

Alternativ gibt es auch: Set.of(E...) und
Map.of(E...)

List.of(E...) ist eine praktische **Factory-Methode**, um schnell eine **Liste mit Elementen** zu erstellen.

```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3);  
    var strings = List.of("1", "2", "3");  
    var persons = List.of(new Person("Sara"), new Person("Jeff"));  
    var personsAsObject = List.<Object>.of(new Person("Sara"), new Person("Anna"));  
  
    System.out.println(numbers.getClass());  
    System.out.println(strings.getClass());  
    System.out.println(persons.getClass());  
    System.out.println(personsAsObject.getClass());  
}
```

List.of(E...)

Alternativ gibt es auch: Set.of(E...) und Map.of(E...)

Achtung: List.of(E...) gibt eine [ImmutableList](#) zurück! Methoden wie add, remove und set werfen eine UnsupportedOperationException!

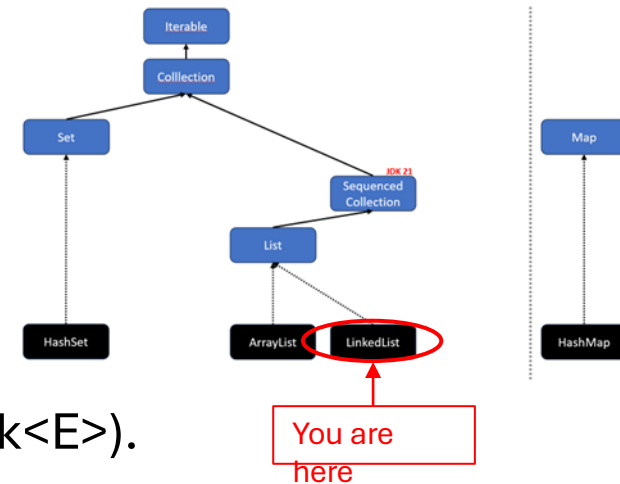
```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3);  
    var strings = List.of("1", "2", "3");  
    var persons = List.of(new Person("Sara"), new Person("Jeff"));  
    var personsAsObject = List.<Object>of(new Person("Sara"), new Person("Anna"));  
  
    System.out.println(numbers.getClass());  
    System.out.println(strings.getClass());  
    System.out.println(persons.getClass());  
    System.out.println(personsAsObject.getClass());  
  
    numbers.add(4);  
}
```

Ausgabe:

```
class java.util.ImmutableCollections$ListN  
class java.util.ImmutableCollections$ListN  
class java.util.ImmutableCollections$List12  
class java.util.ImmutableCollections$List12  
Exception in thread "main" java.lang.UnsupportedOperationException  
    at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.java:142)
```



LinkedList<E>



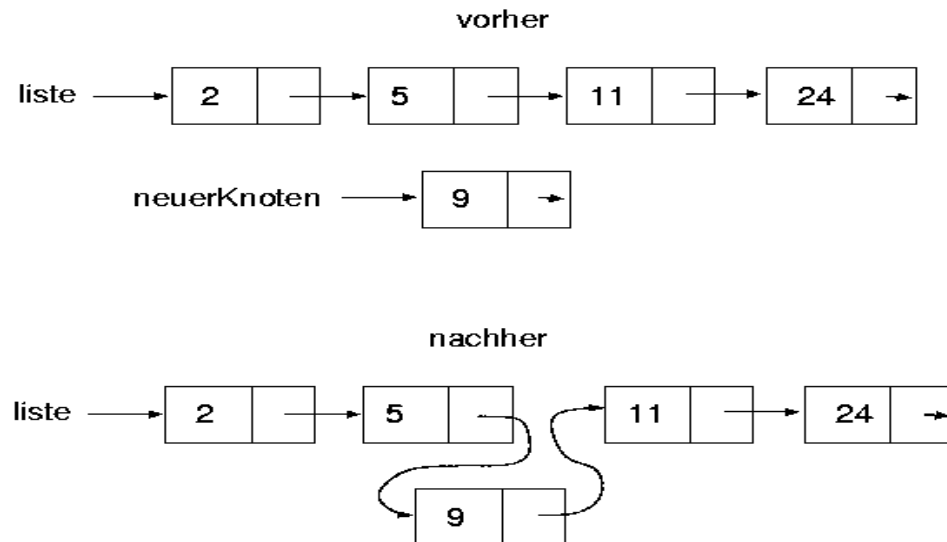
`LinkedList`<E> erweitert das Interface `List`<E> und `Deque`<E> (ehemals `Stack`<E>).

Eine `LinkedList` ist **optimiert für das schnelle Einfügen und Löschen von Elementen**, indem sie intern `Node`-Objekte verwendet, die sich referenzieren.

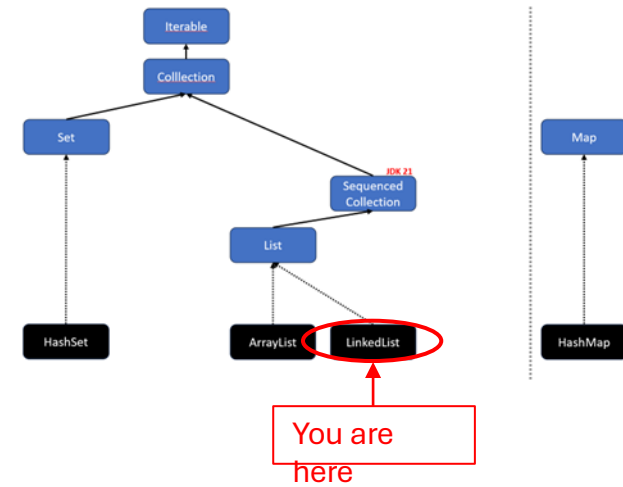
Eine Referenz lässt sich performant durch Zuweisung (`nextNode = newNode;`) „umleiten“.

So kann beim Vergrößern der `LinkedList` ein kostspieliges Umkopieren vermieden werden.

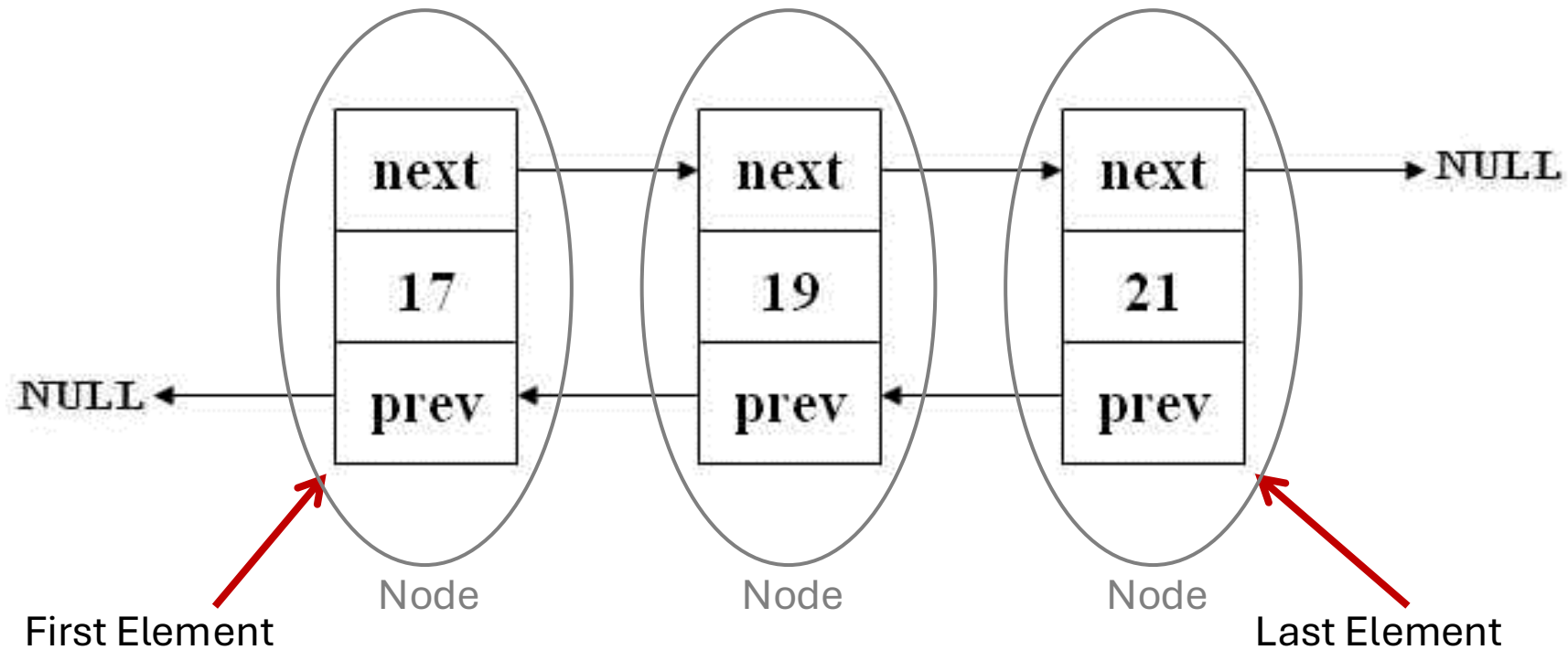
```
var numbers = new LinkedList<Integer>();
numbers.add(2);
numbers.add(5);
numbers.add(11);
numbers.add(24);
numbers.add(index: 2, element: 9);
```



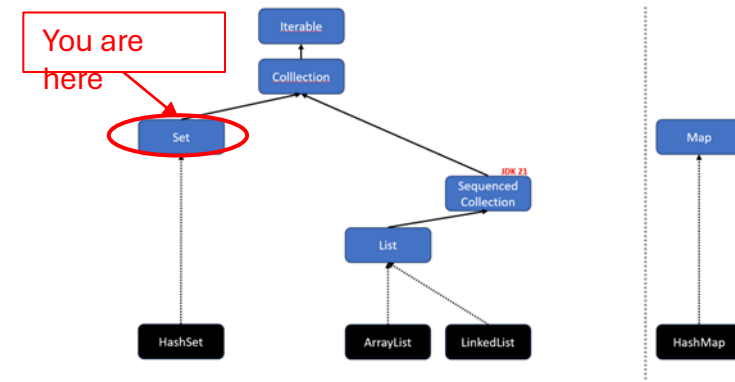
LinkedList<E>



🧐 Java nutzt intern eine doppelt verkettete Liste:



Set<E>



Ein [Set](#)<E> erweitert das Interface `Collection<E>` (es ist also keine `List<E>`).

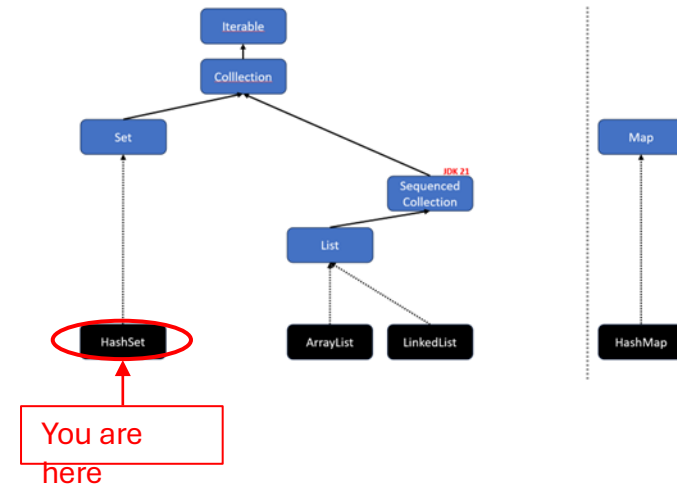
Ein Set entfernt automatisch alle Duplikate und hat keine Ordnung (manchmal).

„A collection that contains no duplicate elements. [...] As implied by its name, this interface models the mathematical *set* abstraction.”

Im Allgemeinen besitzen Sets keine Ordnung. Es gibt allerdings spezielle Implementierungen, die eine Ordnung besitzen:

- `HashSet<E>` Keine Ordnung.
- `LinkedHashSet<E>` Ordnung gemäß der Einfügereihenfolge.
- `TreeSet<E>` Natürliche Ordnung oder eigene Ordnung per [Comparator<T>](#).

HashSet<E>



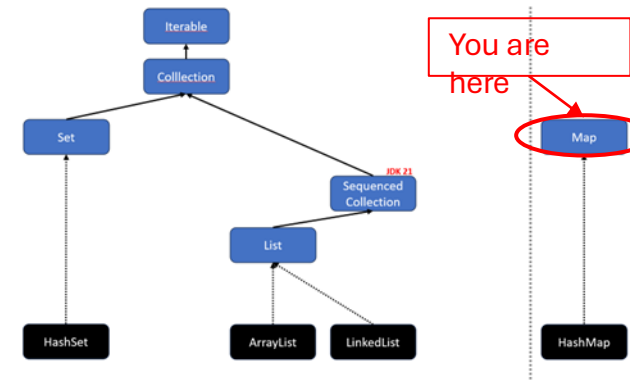
Im Gegensatz zum List-Interface besitzt das Set-Interface keine `get()`-Methode. An die Elemente gelangt man nur mittels Iterator oder for-each.

```
public class Main {
    public static void main(String[] args) {
        Set<Integer> uniqueNumbers = new HashSet<>();
        uniqueNumbers.add(1); // returns true
        uniqueNumbers.add(2); // returns true
        uniqueNumbers.add(2); // returns false
        uniqueNumbers.add(3); // returns true
        for (int number : uniqueNumbers)
            System.out.println(number);
    }
}
```

Ausgabe:

1
2
3

Map<K,V>



Eine Map<K,V> besteht aus Key-Value-Elementen (bzw. Entry).

Ein Entry<K,V> besteht aus einem Schlüssel (Key) und einem Wert (Value).

Schlüssel dürfen nur einmal in einer Map vorkommen, Werte dagegen beliebig oft.

Man kann sich eine Map wie ein flexibleres Array vorstellen...

^{map}products.get(1); VS ^{array}products[1];

...da der Key bei einer Map etwas anderes als ein `int` sein kann (z. B.

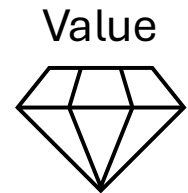
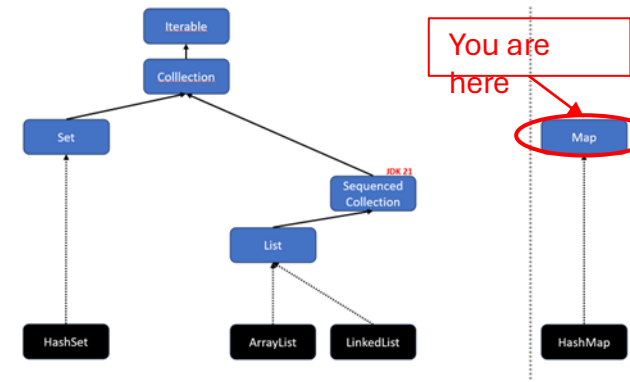


FYI: Maps erweitern in Java nicht das Interface `Iterable<E>` oder `Collection<E>`.
Maps sind trotzdem Teil des Java Collections Frameworks.

String).

Map<K,V>

Key: "Fach14"



```
class Demo {
```

```
    public static void main(String[] args) {  
        Map<String, String> cabinet = new HashMap<>();  
        cabinet.put("Fach14", "Diamond");  
        System.out.println(cabinet.get("Fach14"));  
    }  
}
```

Ausgabe:

Diamond

Map<K,V> – Methoden

// Ein Schlüssel-Wert-Paar hinzufügen
`V put(K key, V value)`

// Prüft, ob der key vorhanden ist
`boolean containsKey(Object key)`

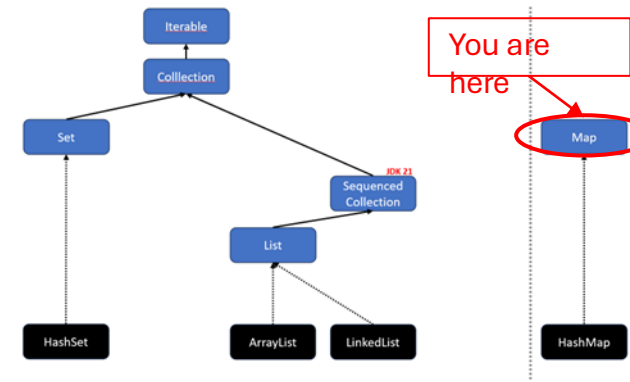
// Einen Wert über seinen Schlüssel bekommen
`V get(Object key)`

// Ein Schlüssel-Wert-Paar entfernen
`V remove(Object key)`

// Ein Set aller Schlüssel der Map erhalten
`Set<K> keySet()`

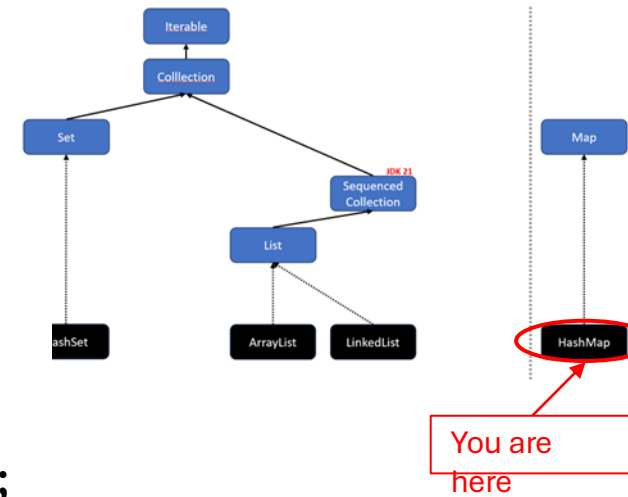
// Eine Collection aller Werte der Map erhalten
`Collection<V> values()`

// Ein Set von Paaren aus Schlüssel und Werten erhalten
`Set<Map.Entry<K, V>> entrySet()`



HashMap<K,V>

```
public class Main {  
    public static void main(String[] args) {  
        Map<String, LocalDate> projectDeadlines = new HashMap<>();  
  
        projectDeadlines.put("PR2", LocalDate.of(2023, 06, 10));  
        projectDeadlines.put("InfC", LocalDate.of(2023, 06, 12));  
  
        System.out.println(projectDeadlines.get("PR2"));  
  
        for (String projectName : projectDeadlines.keySet()) {  
            System.out.println(projectName);  
        }  
  
        for (LocalDate deadline : projectDeadlines.values()) {  
            System.out.println(deadline);  
        }  
    }  
}
```



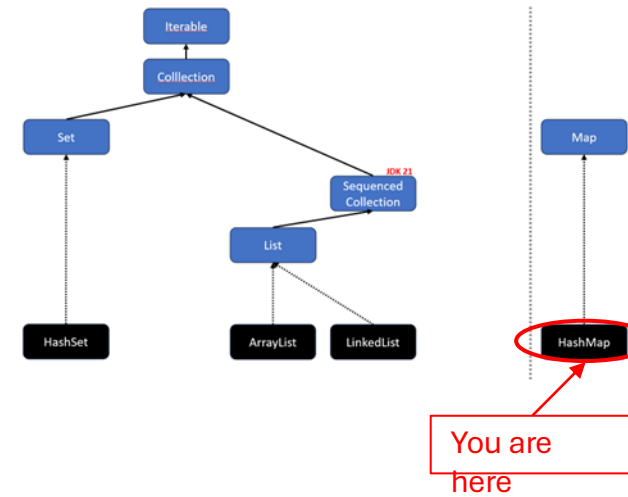
Ausgabe:

```
2023-06-10  
InfC  
PR2  
2023-06-12  
2023-06-10
```

HashMap – EntrySet<K,V>

```
public class Main {  
    public static void main(String[] args) {  
        Map<String, LocalDate> projectDeadlines = new HashMap<>();  
        projectDeadlines.put("PR2", LocalDate.of(2023, 06, 10));  
        projectDeadlines.put("InfC", LocalDate.of(2023, 06, 12));  
  
        for (Entry<String, LocalDate> projectDeadline : projectDeadlines.entrySet()) {  
            System.out.println(projectDeadline.getKey() + "->" + projectDeadline.getValue());  
        }  
    }  
}
```

🧐 FYI: `Entry<String, LocalDate>` kann man hier auch mit `var` ersetzen.



Ausgabe:

```
InfC->2023-06-12  
PR2->2023-06-10
```

λ Lambda

(parameter) -> { anweisung; }

Lambda – Definition

- Ein Lambda-Ausdruck ist ein (kurzer) Codeblock, der
 - optional Parameter entgegennimmt und
 - optional einen Wert zurückgibt.
 - **Parameter und Rückgabewert** werden durch ein **funktionales Interface** festgelegt.
 - **Jedes Lambda basiert auf einem funktionalen Interface.**
- Lambda-Ausdrücke sind ähnlich wie Methoden, sie
 - benötigen **keine Namen** (Implementation ist anonym, Methodename kommt vom Interface),
 - werden **nicht sofort ausgeführt** (sind lazy),
 - sind (möglichst) **zustandslos** (pure),
 - können direkt **innerhalb einer anderen Methode implementiert** werden.

Functional Interfaces

Lambdas basieren in Java auf [Functional Interfaces](#), das sind Interfaces mit **genau einer abstrakten** (genau eine nicht implementierte) Methode.

```
@FunctionalInterface
public interface MeaningOfLife {
    int calculate();
}
```

Lambda mit Parameter

Parameter werden bei Lambdas in den Runden Klammern angegeben. Java kann die Datentypen für die Lambda-Parameter automatisch ermitteln, daher kann man die Datentypen auch weglassen. 🤖

```
@FunctionalInterface
public interface MoneyPrinter {
    void amount(int value);
}

public static void main(String[] args) {
    // Definiere, was das Lambda mit dem Parameter tun soll.
    MoneyPrinter euroPrinter = (int value) -> System.out.print("€" + value);
    MoneyPrinter dollarPrinter = (int value) -> System.out.print("$" + value);

    // Führe das Lambda aus.
    euroPrinter.amount(42);
    dollarPrinter.amount(42);
}
```

Ausgabe: €42 \$42

Lambda – Schreibweisen

Es gibt verschiedene Schreibweisen für Lambdas: mit/ohne runde/geschweifte Klammern.

Variante 1: *parameter* -> *expression* oder { *code block*; }

Beispiel: persons.forEach(person -> System.out.println(person.name));

Beispiel: persons.forEach(person -> { System.out.println(person.name); });

Lambda – Schreibweisen

Es gibt verschiedene Schreibweisen für Lambdas: mit/ohne runde/geschweifte Klammern.

Variante 2: *(parameter) -> expression* oder *{ code block; }*

Beispiel: `persons.forEach((person) -> System.out.println(person.name));`

Beispiel: `persons.forEach((person) -> {
 System.out.println(person.name);
});`

Beispiel: `persons.forEach((Person person) -> System.out.println(person.name));`

Der Parametertyp wird in der Regel weggelassen.
Die Schreibweise mit Parametertyp erfordert runde Klammern.

Lambda – Schreibweisen

Klammern sind nötig, wenn mehr als ein Parameter oder Expression verwendet werden.

Variante 3: *(parameter1, parameter2)*  Ab zwei Parametern sind die runden Klammern () Pflicht. **-> expression oder { code block; }**

Beispiel: `persons.stream().sorted((p1, p2) -> p1.name.compareTo(p2.name));`

Beispiel: `persons.stream().sorted((p1, p2) -> {
 return p1.name.compareTo(p2.name);
});`

 Bei der Kurzschreibweise ohne { } muss man das return weglassen.

Lambda – Schreibweisen

Es gibt eine weitere Schreibweise für Lambdas mit [Methodenreferenz](#).

Variante 4: ***Klassennamen::methodName(parameter)***

Lambda: `numbers.forEach(number -> System.out.println(number));`

Methodenreferenz: `numbers.forEach(System.out::println);`

Lambda: `List.of("a", "b", "c").forEach(letter -> letter.toUpperCase());`

Methodenreferenz: `List.of("a", "b", "c").forEach(String::toUpperCase);`

Lambda: `numbers.forEach(number -> new Person(number));`

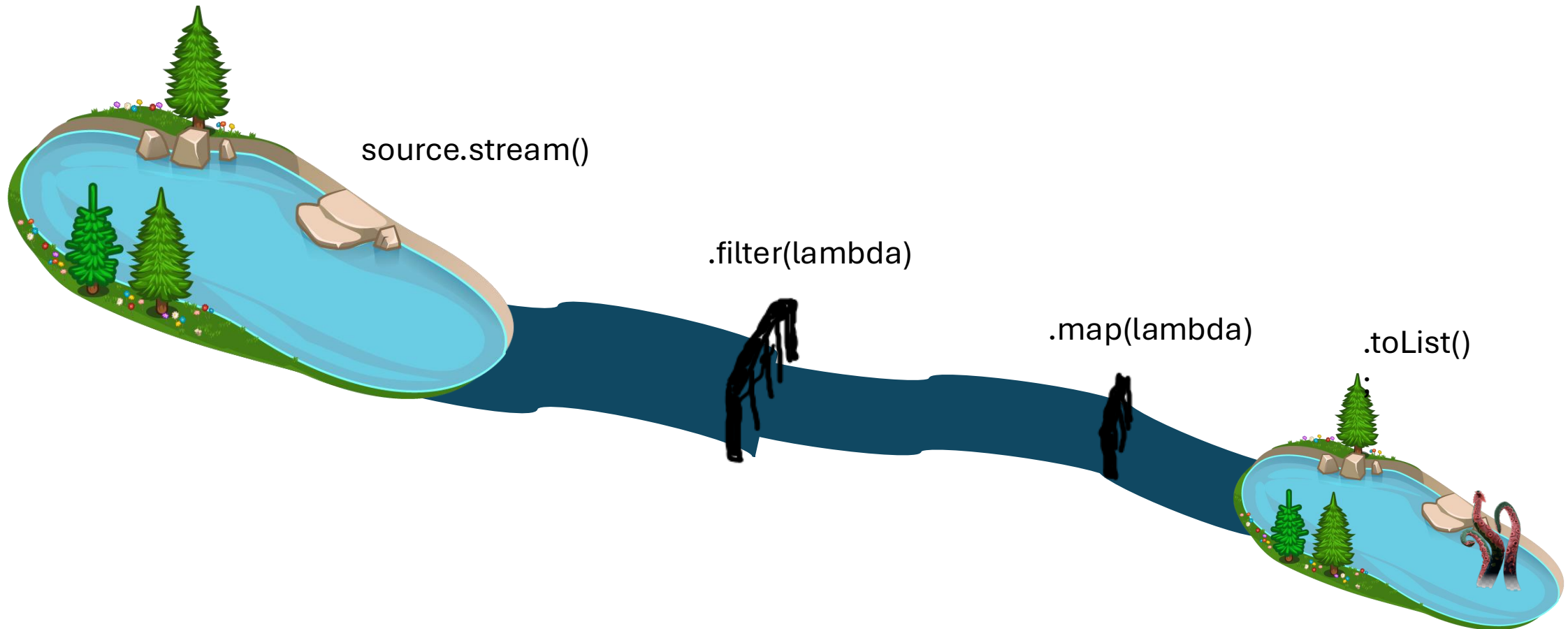
Methodenreferenz: `numbers.forEach(Person::new)`

Lambda: `numbers.stream().sorted((n1, n2) -> n1.compareTo(n2));`

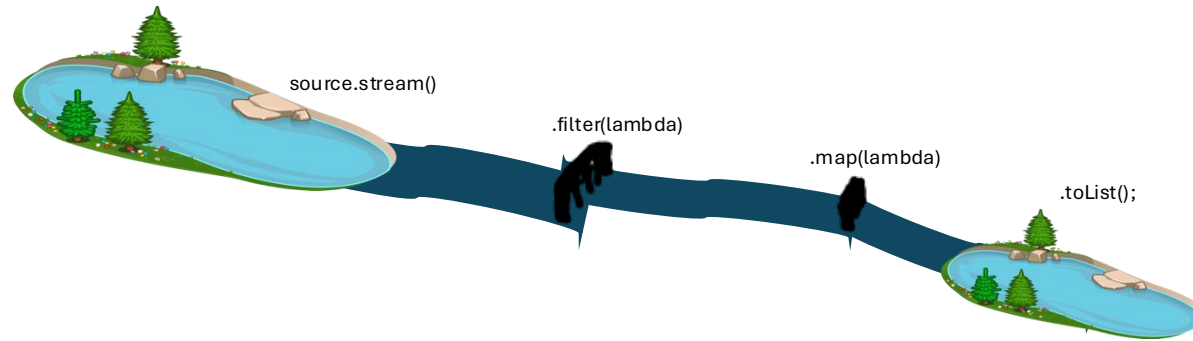
Methodenreferenz: `numbers.stream().sorted(Integer::compareTo);`

Stream

Das Interface [Stream<T>](#) aus `java.util.stream` beschreibt einen „Strom“ von Elementen. Dieser „fließt“ von einer Quelle zu einem Ziel. Auf seinem Weg kann der Strom nahezu beliebig manipuliert werden.



Stream



Stream-Erstellung

- `collection.stream()`
- `Stream.of(...)`
- ...

Intermediate Operatoren

- `.filter(lambda)`
- `.map(lambda)`
- `.flatMap(lambda)`
- `.sorted(lambda)`
- ...

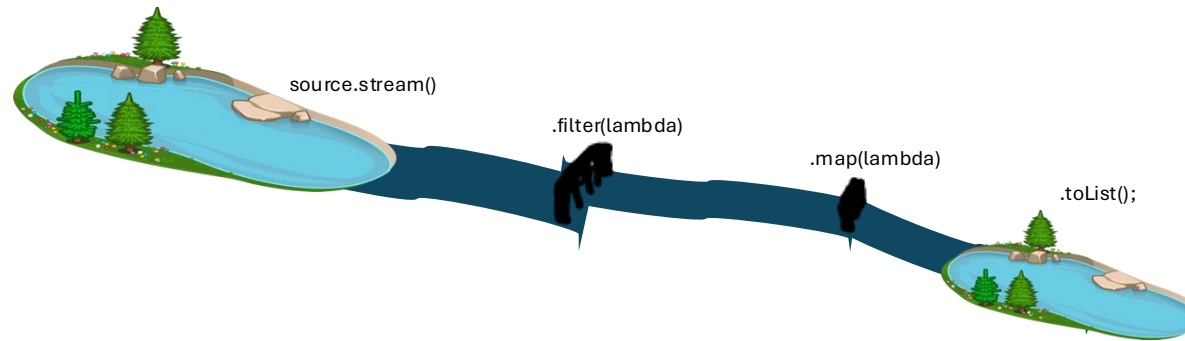
Terminale Operatoren

- `.toList()`
- `.collect(...)`
- `.toList()`
- `.sum()`
- `.reduce()`
- ...



```
Stream.of(3, 2, 1).filter(i -> i>1).sorted().toList();
```

Stream



Stream-Erstellung

- collection.stream()
- Stream.of(...)
- ...

Intermediate Operatoren

- .filter(lambda)
- .map(lambda)
- .flatMap(lambda)
- .sorted(lambda)
- ...

Terminale Operatoren

- .toList()
- .collect(...)
- .toList()
- .sum()
- .reduce()
- ...



```
Stream.of(3, 2, 1).filter(i -> i>1).sorted().toList();
```



toList() gibt eine nicht veränderbare List zurück (siehe [unmodifiable in Collections](#)).

D. h. die Ergebnisliste kann nicht verändert werden z. B. durch add, remove, set (UnsupportedOperationException)!



Stream – Essentielles Wissen

- Ein Stream selbst **speichert keine Elemente**. Er bezieht sie aus einer **Quelle**. Typische Quellen sind Collections (z. B. ArrayList, HashSet, Arrays), aber auch andere Quellen sind möglich.
- Nach der Erzeugung eines Streams, können **beliebig** viele **intermediate Operationen** zum Stream hinzugefügt werden.
- Ein Stream ist im Prinzip eine **Sammlung von auszuführenden Funktionen**.
- Streams sind **lazy**. Die enthaltenen **Funktionen** werden erst **durch Aufruf einer terminalen Operation nacheinander ausgeführt**.
- Ein Stream wird durch eine terminale Operation geschlossen.
Der Stream kann anschließend **nicht wiederverwendet** werden!

Stream – Intermediate Operations

Intermediate Operationen **geben immer `Stream<T>` zurück**. Eine vollständige Liste findest du [hier](#).

<code>Stream<T> filter(lambda)</code>	Liefert die Elemente zurück, die das vorgegebene Prädikat erfüllen.
<code>Stream<T> map(lambda)</code>	Mache etwas mit jedem Element und gebe die modifizierten Elemente zurück.
<code>Stream<T> flatMap(lambda)</code>	„Liste von Listen“ zu Liste: <code>[[1, 2, 3], [4, 5], [6, 7, 8]] → [1, 2, 3, 4, 5, 6, 7, 8]</code> .
<code>Stream<T> sorted(lambda)</code>	Sortiere die Liste (siehe Interface Comparator<T>)
<code>Stream<T> distinct()</code>	Entferne alle Duplikate.
<code>Stream<T> limit(long maxSize)</code>	Gebe die ersten x Elemente zurück.
<code>Stream<T> skip(long n)</code>	Überspringe die ersten x Elemente.
<code>Stream<T> boxed()</code>	Wandelt einen primitiven Stream zu einem Objekt-Stream um.
<code>Stream<T> peek(lambda)</code>	Zum Debuggen, um sich Zwischenergebnisse eines Streams anzuschauen.

Stream – Terminal Operations

Terminal Operations geben unterschiedliche Werte zurück, je nach Methode. Eine vollständige Liste findest du [hier](#).

List<T> toList()	Gebe alle Elemente des Streams als <i>unmodifiable</i> List zurück.
T[] toArray (T[]::new)	Gebe alle Elemente des Streams als Array vom Typ T zurück.
Collection<T> collect (lambda)	Ermöglicht mutable reduction - und Collectors -Operationen (z. B. teeing, joining, groupingBy).
T/Optional<T> reduce (lambda)	Reduziert Elemente zu einem Element (z. B. Summe aller Zahlen).
void forEach (lambda)	Iteriere über jedes Element (wie for-Schleife). Kein Rückgabewert!!!
long count ()	Gebe die Anzahl der Elemente zurück.
boolean anyMatch (lambda)	Gebe true zurück, wenn irgendein Element die gegebene Bedingung erfüllt.
boolean allMatch (lambda)	Gebe true zurück, wenn alle Elemente die gegebene Bedingung erfüllen.
boolean noneMatch (lambda)	Gebe true zurück, wenn kein Element die gegebene Bedingung erfüllt.
Optional<T> findAny (lambda)	Gibt ein zufälliges Element zurück.
Optional<T> findFirst (lambda)	Gibt das erste Element zurück.
Optional<T> min (lambda)	Gebe das kleinste Element anhand des gegebenen Comparator<T> zurück.
Optional<T> max (lambda)	Gebe das größte Element anhand des gegebenen Comparator<T> zurück.

Test-Driven Development (TDD)

„Test First“

Schreibe erst den Testfall, dann den zugehörigen Programm-Code.

Vorteile:

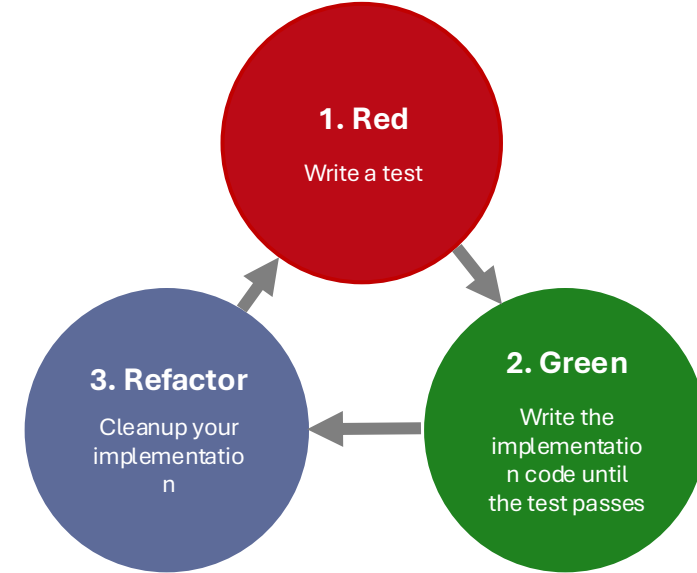
- Man macht sich vorab strukturiert Gedanken, welche Rückgabewerte und Parameter ein Feature benötigt, um es besser testen zu können.
- Die Anforderungen für das Feature lassen sich über die Tests vorab definieren.

Nachteile:

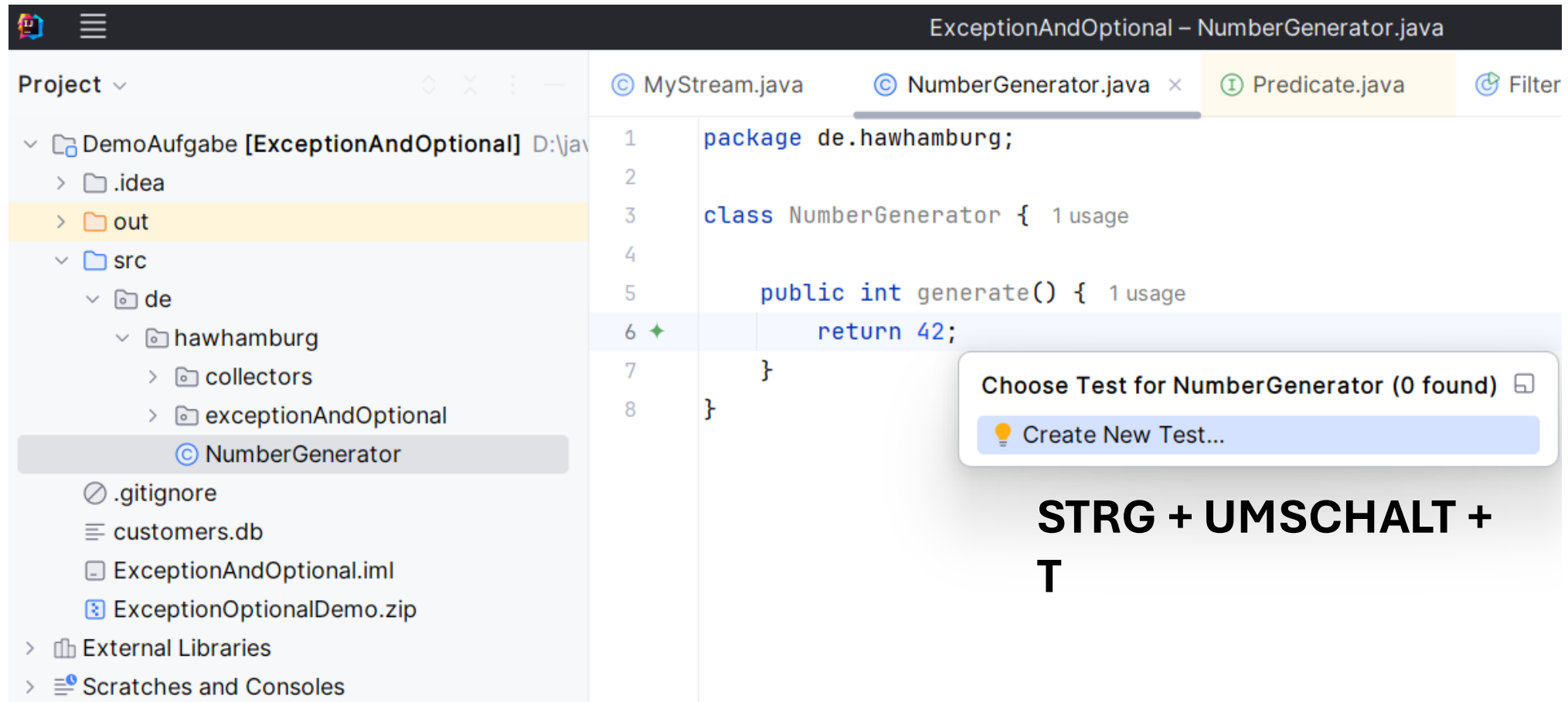
- Führt unter Umständen zu kleinteiligen Schritten und Refactorings, die übersprungen werden können.

“According to our findings, the benefits of test-driven development compared to iterative test-last development are small and thus in practice relatively unimportant, although effects are positive. There is an indication of test-driven development endorsing better branch coverage, but effect size is considered small.”

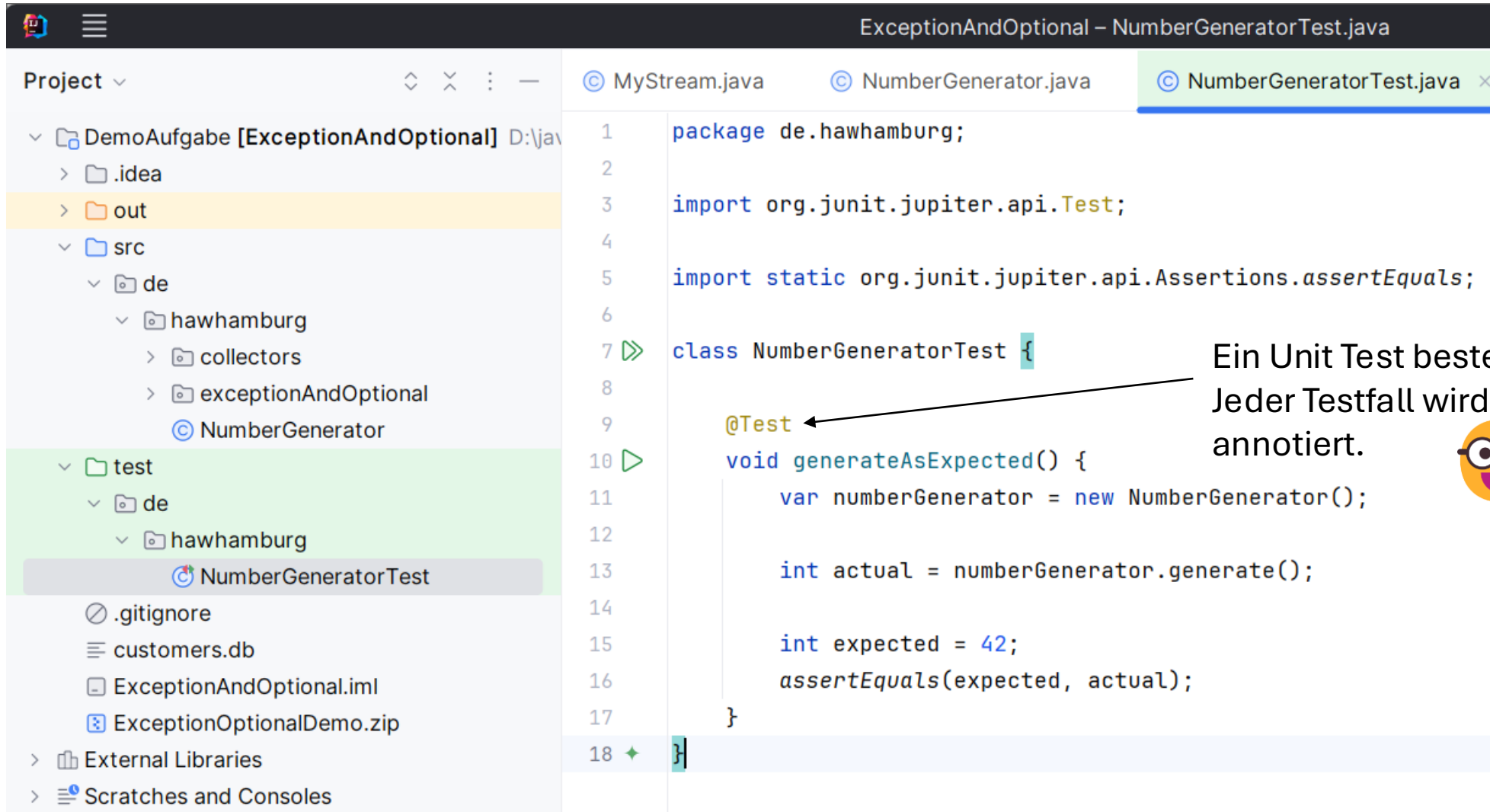
Quelle: <https://www.sciencedirect.com/science/article/abs/pii/S0950584911000346>



Unit Test mit IntelliJ erstellen



Unit Test in IntelliJ



The screenshot shows the IntelliJ IDEA interface. On the left, the 'Project' view displays the directory structure: `DemoAufgabe [ExceptionAndOptional]` contains `.idea`, `out`, `src` (with `de` containing `hawhamburg` and `exceptionAndOptional`), and `test` (with `de` containing `hawhamburg` and `NumberGeneratorTest`). The `NumberGeneratorTest` file is selected. The main editor shows the code for `NumberGeneratorTest.java`:

```
1 package de.hawhamburg;
2
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.assertEquals;
6
7 class NumberGeneratorTest {
8
9     @Test
10     void generateAsExpected() {
11         var numberGenerator = new NumberGenerator();
12
13         int actual = numberGenerator.generate();
14
15         int expected = 42;
16         assertEquals(expected, actual);
17     }
18 }
```

An arrow points from the text on the right to the `@Test` annotation on line 9.

Ein Unit Test besteht aus Testfällen.
Jeder Testfall wird mit `@Test`
annotiert.



JUnit 5 – Asserts und Annotations

Assert-Methoden:

z. B. *assertEquals*, *assertTrue*, *assertFalse*, *assertNull*,
assertNotNull, *assertThrows*, *assertAll*, *assertIterableEquals*,
assertArrayEquals, *assertLinesMatch*, *assertSame*, *assertNotSame*,
assertTimeout, *fail*

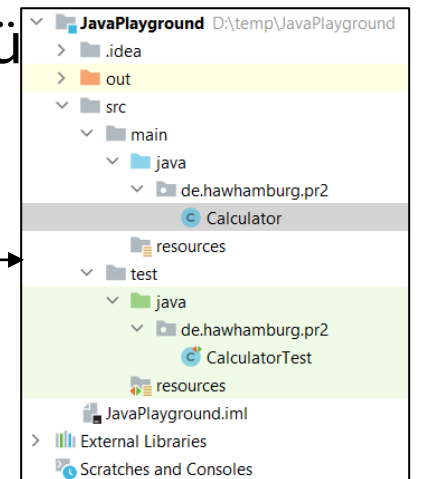
Annotationen:

z. B. *@BeforeAll*, *@BeforeEach*, *@Test*, *@AfterEach*, *@AfterAll*,
@RepeatedTest(int)

Siehe auch: <https://junit.org/junit5/docs/current/user-guide/>

Best Practices 🧐

1. JUnit: erwarteter Wert kommt zuerst: `assertEquals(expected, actual)`; Werden die Werte vertauscht, erhält man verwirrende Fehlermeldungen.
2. Benenne Testfälle möglichst prägnant: was wird getestet und warum?
3. Nutze das AAA-Pattern.
4. Testfälle müssen unabhängig von anderen Testfällen sein.
5. Teste ein Konzept pro Testfall: möglichst ein `assert` pro Methode.
6. Pushe niemals Code mit fehlgeschlagenen Tests ins Git-Repository.
7. Unit-Tests sollen möglichst schnell sein (damit man sie häufig prüft).
8. Habe eine hohe Testabdeckung inkl. Randfälle (Edge-Cases).
9. Verwende Äquivalenzklassen.
10. Verwende die Standard-Ordnerstruktur von Apache.
11. Verwende Erweiterungen wie AssertJ, Truth oder Hamcrest.



Best Practices: AAA-Pattern



Innerhalb der Testfälle wird das AAA-Schema empfohlen oder alternativ given-when-then:

Arrange (given)

Initialisiere die benötigten Testdaten.

Act (when)

Führe die zu testende Funktionalität aus.

Assert (then)

Prüfe, ob das Ergebnis bzw. Verhalten mit der Erwartung übereinstimmt.

Best Practices: AAA-Pattern Beispiel



```
class StudentFinderTest {  
  
    @Test  
    void findStudentsByUniversity() {  
        // Arrange (given)  
        ArrayList<Student> expected = new ArrayList<>(List.of(  
            new Student("Grace Hopper", "HAW Hamburg"),  
            new Student("Robert Cecil Martin", "HAW Hamburg")  
        ));  
  
        // Act (when)  
        List<Student> actual = new StudentFinder().findByUniversity("HAW Hamburg");  
  
        // Assert (then)  
        assertEquals(expected, actual);  
    }  
}
```

Best Practices: Wie viele Asserts?

Teste nur ein Konzept pro Testfall, d. h., halte die Anzahl an *asserts* pro Testfall möglichst gering! Die asserts sollten pro Testfall inhaltlich zusammengehören, z. B. erst die Anzahl der Elemente in einer Liste prüfen und anschließend die Werte der enthaltenen Elemente prüfen.

Das sind zwei unterschiedliche Konzepte bzw. Testfälle.

@Test

```
void twoPositiveNumbersAreCorrectlyAddedAndSubtracted() {
```

```
    // Arrange
```

```
    Calculator calculator = new Calculator();
```

```
    int numberA = 1;
```

```
    int numberB = 0;
```

```
    int expectedResult = 1;
```

```
    // Act
```

```
    int actualAddResult = calculator.add(numberA, numberB);
```

```
    int actualSubResult = calculator.subtract(numberA, numberB);
```

```
    // Assert
```

```
    assertEquals(expectedResult, actualAddResult);
```

```
    assertEquals(expectedResult, actualSubResult);
```

```
}
```

Teste die Subtraktion in einem separaten Testfall:

@Test

```
void twoPositiveNumbersAreCorrectlySubtracted() {
```

```
    int expectedResult = 1;
```

```
    int actualSubResult = new Calculator().subtract(1, 0);
```

```
    assertEquals(expectedResult, actualSubResult);
```

```
}
```

Best Practices: Edge Cases

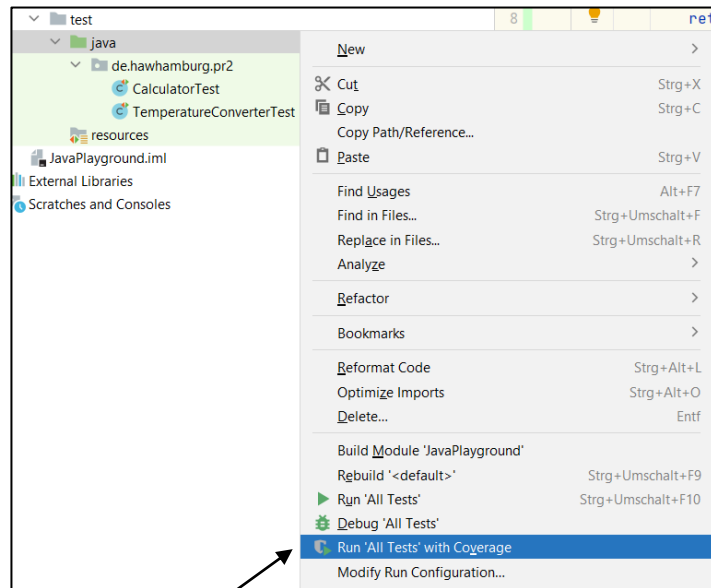


Randfälle bzw. „Edge Cases“ sind Werte oder Eingaben, die am Rande des zulässigen Bereichs liegen oder außergewöhnliche Umstände widerspiegeln, z. B.:

- **null, 0** und **negative Werte**.
- **Leere Listen**, Listen mit nur einem Element, Listen mit vielen Elementen.
- **Leere Eingaben** (z. B. leerer String) oder Eingaben mit **minimaler** Länge (z. B. `""`).
- **Lange Eingaben** oder Eingaben mit **maximaler** Länge.
- **Sonderzeichen** oder ungültige Zeichen.
- **Falscher Datentyp** bzw. unerwartete Formate oder Strukturen.
- **Zeitberechnungen** wie Schaltjahre oder Sommer-/Winterzeit.
- Mathematische Berechnungen wie **Division durch 0**.
- **Überlauf** von Zahlen, z. B.: $(2147483647 + 1) \rightarrow -2147483648$.
- **Exceptions**.

Best Practices: Test Coverage 1/4

aka „Testabdeckung“



1. Test Coverage starten.

```
1 package de.hawhamburg.pr2;
2
3 import java.security.InvalidParameterException;
4
5 3 usages
6 public class TemperatureConverter {
7     1 usage
8     public double convert(double value, String convertToUnit) {
9         if ("Fahrenheit".equals(convertToUnit)) {
10             return (value * (9d / 5d)) + 32;
11         } else if ("Celsius".equals(convertToUnit)) {
12             return (value - 32) * (5d / 9d);
13         } else {
14             throw new InvalidParameterException("Temperaturumwandlung ist nicht möglich.");
15         }
16     }
17 }
```

2. Test Coverage Ergebnisse.

Element	Class, %	Method, %	Line, %
de.hawhamburg.pr2	100% (2/2)	100% (2/2)	62% (5/8)
Calculator	100% (1/1)	100% (1/1)	100% (2/2)
TemperatureConverter	100% (1/1)	100% (1/1)	50% (3/6)

Nicht alle Bedingungen der if-Anweisung werden getestet!
Die Testabdeckung sollte möglichst hoch sein.

```
class TemperatureConverterTest {
    @Test
    void givenCelsius_whenConvert_thenReturnFahrenheit() {
        var temperatureConverter = new TemperatureConverter();
        String conversionOption = "Fahrenheit";
        var celsius = 10;
        var expectedFahrenheit = 50;

        var actualFahrenheit = temperatureConverter.convert(celsius, conversionOption);

        assertEquals(expectedFahrenheit, actualFahrenheit);
    }
}
```

Wie hoch???
Darüber wird gestritten...

Best Practices: Test Coverage 2/4

1 **Line Coverage:**

2
3 Misst den Anteil der Codezeilen, die während der Ausführung der Tests
4 mindestens einmal ausgeführt werden.
5

Branch Coverage:

Misst den Anteil der Verzweigungen (z. B. in if-else-Anweisungen, Anweisungen), die während der Tests alle möglichen Ausgänge haben.

→ Branch Coverage ist aufwendiger, sollte aber präferiert werden.



Best Practices: Test Coverage 3/4

```
boolean isOfLegalAge(Integer age) {  
    if (age != null && age >= 18) {  
        return true;  
    }  
    return false;  
}
```

100% Line Coverage:

Alle Zeilen werden abgedeckt.

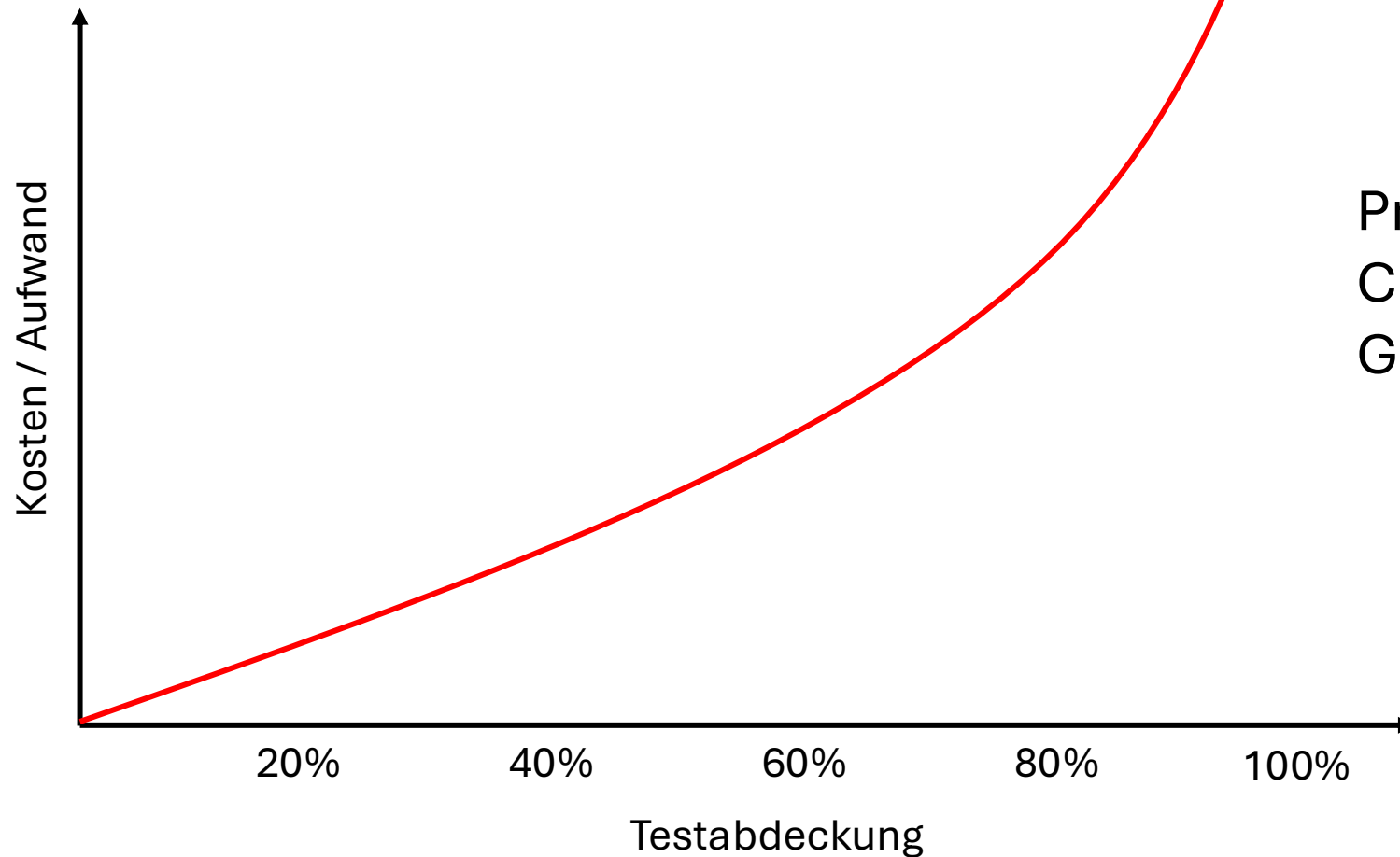
D. h. jeweils ein Testfall für `isOfLegalAge(17)` und `isOfLegalAge(18)`.

100% Branch Coverage:

Alle Pfade werden abgedeckt.

D. h. jeweils ein Testfall für `isOfLegalAge(17)`, `isOfLegalAge(18)` und `isOfLegalAge(null)`.

Best Practices: Testcoverage 4/4



Priorisiere Tests für kritischen Code anstatt z. B. generierte Getter/Setter zu testen.

Tests: Fragen

- Welche Arten von Test Coverage gibt es?
- Was heißt dabei 100%
- Zählen da auch Abdeckungen wie Nullpointer dazu?
 - Typische Edge Cases testen wenn angebracht (z. B. null, negative Zahlen, leere Liste, Duplikate, Sortierung)

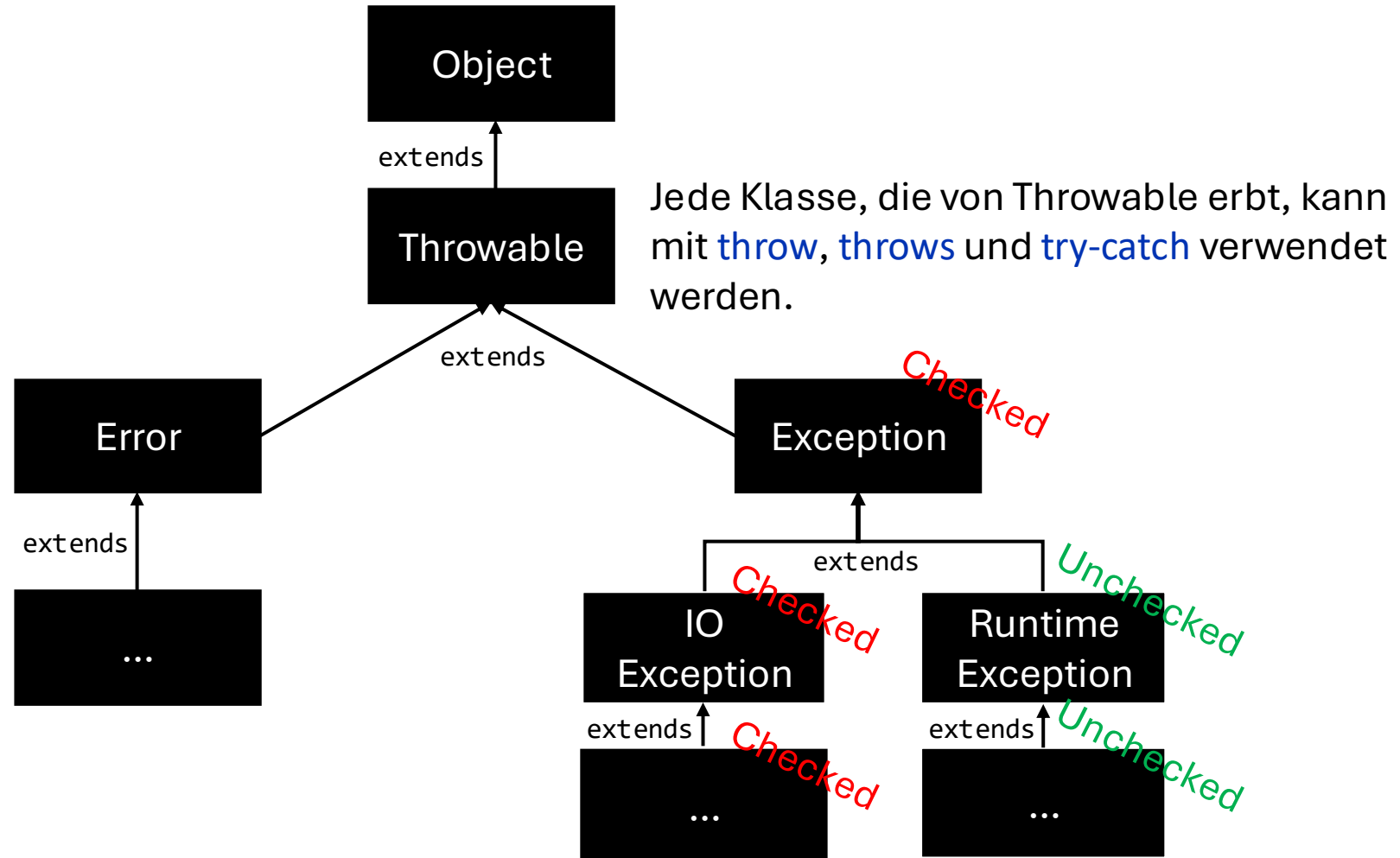
Wie eine Klausuraufgabe formuliert sein **könnte**:

Schreibe einen Unit Test für diese Klasse mit 100%iger Testabdeckung (branch coverage).

Ggf. auch: Teste auch mögliche Edge-Cases.

Exceptions in Java

"Ein Error weist auf ernsthafte Probleme hin, die eine sinnvolle Anwendung nicht abfangen sollte. Error finden unter abnormalen Bedingungen statt, die niemals auftreten sollten." Beispiel: Hardwarefehler.



Checked oder Unchecked Exception?

Java basiert auf der *Java Virtual Machine* (JVM) und ist eine der wenigen Programmiersprachen, die Checked Exceptions unterstützt. Weitere Sprachen mit Checked Exceptions sind Kotlin (JVM), Scala (JVM) und Objective-C.

Checked Exceptions haben sich kaum in anderen Programmiersprachen etabliert, da oft die Flexibilität von Unchecked Exceptions bevorzugt werden.

Globales Exception Handling?

Frage: Warum nicht alle Exceptions global "ganz oben" in der main-Methode abfangen?

```
public static void main(String[] args) {  
    try {  
        // start application  
        new MainView().show();  
        // run app ...  
    } catch (Exception e) {  
        System.err.println(e.getMessage());  
    }  
}
```

Antwort: In der Regel möchte man eine Exception möglichst lokal abfangen und behandeln (Fail Fast Prinzip), z. B. weil der Fehler im Dialog-Fenster angezeigt werden soll und nicht im Hauptfenster. Es ist zudem oft sinnvoller, sofern möglich, Exceptions lokal zu behandeln, damit sie nicht überall im Softwaresystem auftauchen.

Darüber hinaus sind Exceptions langsam im Vergleich zu if-else-Abfragen, da beim Werfen einer Exception jedes Mal ein Call-Stack (Historie der Methodenaufrufe) erzeugt wird.

Fail Fast Principle



Je später Fehler behandelt werden, **desto schwieriger** lässt sich die **Fehlerursache** im Nachhinein ermitteln.

Aus diesem Grund besagt das Fail Fast Principle:

- Fehler so früh wie möglich erkennen.
- Fehler so früh wie möglich zurückmelden.
- Fehler möglichst lokal behandeln und nicht weitergeben.

Ziel: Einfache Diagnose und Debugging.

Fail Fast Principle: Beispiel (Preconditions)

Prüfe **zu Beginn** einer **Methode** oder eines **Konstruktors**, ob die Parameter **valide** sind.
Beispiel:

```
Optional<Invoice> calculateByCustomer(String name) {  
    // 1. do validity checks first (preconditions).  
    if(name == null || name.isBlank()) {  
        throw new IllegalArgumentException(name); // or use Optional<T> instead of Exception (see upcoming slides).  
    }  
  
    // 2. then do the actual operation with valid parameters.  
    // query customer by name...  
}
```

Die Prüfung am Anfang der Methode nennt man auch **Precondition**: die Methode prüft zunächst alle Parameter auf Gültigkeit. Ist ein Parameter ungültig, wird die Methode sofort mit einer Exception abgebrochen, da eine Vorbedingung verletzt wurde und die Methode nicht korrekt ausgeführt werden könnte.

null



Null Problemo



`null` sollte möglichst vermieden werden, da sonst überall im Code `null`-Checks erforderlich sind:

```
class InvoiceManager {
```

```
    Invoice calculateByCustomer(String name) {
```

```
        var customer = database.findCustomer(name);
```

```
        if (customer != null) {
```

```
            // do calculations with customer and fill result...
```

```
            return new Invoice(42.50);
```

```
        } else {
```

```
            return null; // Der Aurufer von calculateByCustomer muss wieder einen null-Check für die Invoice machen... 🤪
```

```
        }
```

```
    }
```

```
}
```

Primitive statt null

Primitive (`byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, `char`) sind, soweit möglich, vorzuziehen, da diese niemals `null` sein können. 👍

`int number = null;` // Nicht kompilierbar. Nur Objekte können null sein.

Objekte können dagegen `null` sein, wie z. B. bei den Wrapper-Klassen Integer, Double, Character usw.

```
class MathUtil {  
    static double square(double number) { // Rückgabewert und Parameter kann niemals null sein, kein null-Check notwendig!  
        return number * number;  
    }  
  
    static Double square(Double number) { // Rückgabewert und Parameter kann null sein, null-Check notwendig!  
        return number * number;  
    }  
  
    static void main(String[] args) {  
        System.out.println(square(null)); // Der Compiler wählt hier die Double-Methode aus, weil nur diese Methode null-Werte akzeptiert.  
    }  
}
```

Exception statt null

Eine Möglichkeit, `null` zu vermeiden, wäre eine `RuntimeException` zu werfen.

```
interface Database {  
    Customer findCustomer(String name) throws DatabaseException; // DatabaseException erbt von RuntimeException  
}  
  
class InvoiceManager {  
    public Invoice calculateByCustomer(String name) {  
        var customer = database.findCustomer(name); // wirft eine RuntimeException.  
        // do calculations with customer and fill result...  
        return new Invoice(42.50);  
    }  
}
```

Vorteil: Die `RuntimeException` kann ignoriert werden.

Nachteil: Die Exception ist "versteckt". Sie ist häufig nur im JavaDoc dokumentiert und kann daher übersehen werden. Das kann dazu führen, dass ein `try-catch` vergessen wird (lokal oder global), und die App zur Laufzeit abstürzen kann. Das sollte nicht passieren!

Warum muss ich beides prüfen?

```
if (numbers == null || numbers.isEmpty()) { ... }
```

null und "leer" sind zwei völlig verschiedene Zustände sind, die unterschiedliche Probleme verursachen.

numbers == null

- **Was es prüft:** Diese Prüfung schaut, ob die Variable numbers überhaupt auf ein Objekt verweist.
- **Was ist null?** null bedeutet "nichts". Die Variable existiert, aber sie zeigt nicht auf eine Kiste (Collection) im Speicher. Sie zeigt ins Leere.
- **Warum ist das ein Problem?** Wenn numbers null ist und du versuchst, eine Methode darauf aufzurufen (wie .isEmpty() oder .size()), stürzt dein Programm sofort mit einer NullPointerException ab.
- **Analogie:** Du fragst, ob du überhaupt eine Kiste bekommen hast.

numbers.isEmpty()

- **Was es prüft:** Diese Prüfung schaut, ob die Collection, auf die numbers verweist, irgendwelche Elemente enthält.
- **Was ist "leer"?** "Leer" bedeutet, dass du ein *echtes, valides Collection-Objekt* hast (es ist nicht null), aber es sind einfach null Elemente darin. Die Größe ist 0.
- **Warum ist das ein Problem?** Für *diese spezifische Methode* (calculateAverageOptional) ist eine leere Liste ein Problem. Du kannst keinen Durchschnitt aus null Zahlen berechnen (das würde zu einer Division durch null führen). In diesem Fall ist es logisch korrekt, ein Optional.empty() zurückzugeben.
- **Analogie:** Du hast eine Kiste bekommen, aber sie ist leer.

Reihenfolge

- Java wertet den Ausdruck `A || B` von links nach rechts aus:
- **Fall 1: numbers ist null**
 - Java prüft `numbers == null`.
 - Das Ergebnis ist `true`.
 - Wegen des `||` (ODER) reicht ein `true` aus, damit die gesamte Bedingung wahr ist. Java **hört sofort auf zu prüfen** und führt den Code im `if`-Block aus.
 - **Wichtig:** `numbers.isEmpty()` wird **niemals aufgerufen**. Dadurch wird die `NullPointerException` verhindert!
- **Fall 2: numbers ist eine leere Liste (nicht null, aber leer)**
 - Java prüft `numbers == null`.
 - Das Ergebnis ist `false`.
 - Da der erste Teil `false` war, *muss* Java jetzt den zweiten Teil prüfen: `numbers.isEmpty()`.
 - Dieser Aufruf ist **sicher**, da wir ja wissen, dass `numbers` nicht `null` ist.
 - `numbers.isEmpty()` ergibt `true`.
 - Die Gesamtbedingung (`false || true`) ist `true`, und der `if`-Block wird ausgeführt (was korrekt ist, da wir keinen Durchschnitt berechnen können).
- **Fall 3: numbers ist eine gefüllte Liste**
 - Java prüft `numbers == null`. Das ist `false`.
 - Java prüft `numbers.isEmpty()`. Das ist auch `false`.
 - Die Gesamtbedingung (`false || false`) ist `false`. Der `if`-Block wird übersprungen, und deine Methode kann den Durchschnitt berechnen

Zusammenfassung:

- Du prüfst auf **null**, um einen Programmabsturz (NullPointerException) zu verhindern.
- Du prüfst auf **isEmpty()**, um einen logischen Fehler (wie Division durch null) zu verhindern und den Anwendungsfall "keine Daten" korrekt zu behandeln (indem du z.B. Optional.empty() zurückgibst).

Optional<T>



Optional<T> ist eine Wrapper-Klasse: Ein Optional<T>-Objekt enthält ein anderes Objekt. Im Grunde funktioniert ein Optional<T> wie eine Collection<T> mit maximal einem Element (Optional<T> ist allerdings nicht Iterable).

Mit Optional<T> lässt sich in der Programmiersprache ausdrücken, dass ein Objekt eventuell nicht vorhanden ist (anstatt **null** zu verwenden).

Durch die `ifPresent`- und `orElse`-Methoden in Optional<T> kann das enthaltene Objekt herausgeholt und gleichzeitig eine Alternative codiert werden, falls das Objekt nicht vorhanden (**null**) ist. Dadurch lassen sich if-else-Abfragen vermeiden. 🤖

Optional<T> erzeugen

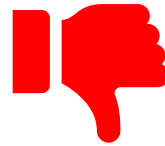
Es gibt in der Klasse Optional<T> folgende statische Factory-Methoden, um ein Optional-Objekt zu erzeugen:

1. `return Optional.empty();` *// ein leeres Optional-Objekt, quasi der Ersatz für null*
2. `return Optional.of(customer);` *// wirft NullPointerException wenn customer == null*
3. `return Optional.ofNullable(customer);` *// i. d. R. die Methode der Wahl.
Falls customer == null, return empty Optional, sonst ein Optional mit dem customer-Objekt.*

Optional<T> verwenden

Einige Methoden aus [Optional<T>](#):

~~boolean~~ isEmpty() // *möglichst vermeiden*
~~boolean~~ isPresent() // *möglichst vermeiden*
T get() // *möglichst vermeiden*



T orElse(T other)
T orElseGet(lambda)
T orElseThrow()
T orElseThrow(lambda)



void ifPresent(lambda)
void ifPresentOrElse(lambda, lambda)

Optional<U> map(lambda) // *Optional<T> kann .stream() und somit alle Stream-Methoden.* 🤖

Optional<T> or...



Man gewinnt mit den Optional-Methoden `isPresent` und `get` nichts – man hätte genauso gut `null` zurückgeben können statt ein `Optional`, weil man wieder eine `if`-Abfrage macht. Beispiel:



// null-Variante:

```
Customer customer = db.findCustomer(name);  
if (customer != null) { // null-Check  
    System.out.println(customer.name());  
} else {  
    System.out.println("NoName");  
}
```



// Optional-Variante mit isPresent und get:

```
Optional<Customer> customer = db.findCustomer(name);  
if (customer.isPresent()) { // quasi null-Check  
    System.out.println(customer.get().name());  
} else {  
    System.out.println("NoName");  
}
```



// orElse gibt entweder den Wert im Optional zurück, wenn dieser existiert; falls nicht, wird ein Default-Wert "NoName" zurückgegeben.
// Optional unterstützt auch die Stream-Methode map. Diese wird hier genutzt, um den name() zu extrahieren, sofern der customer existiert.
System.out.println(customer.map(Customer::name).orElse("NoName"));

Optional<T> statt **null**

Optional<T> sollte **als Rückgabewert** verwendet werden, falls das Objekt **null** sein könnte.

- Mit Optional kann **explizit in der Sprache ausgedrückt** werden, dass ein Objekt eventuell nicht vorhanden (**null**) sein könnte.
- Mit Optional<T> lässt sich bereits zur Kompilezeit **null** isolieren und damit NullPointerException vermeiden.

Hinweis: Die Verwendung von Optional<T> für Parameter ist umstritten.

Besser: Methoden-Overloading oder [Varargs...](#)

"API Note: Optional is primarily intended for use as a method return type where there is a clear need to represent "no result," and where using null is likely to cause errors. A variable whose type is Optional should never itself be null; it should always point to an Optional instance."

Wann Exception verwenden?

- Exceptions sind in der Regel **langsam in Bezug auf Performanz**.
- Exceptions sind für **außergewöhnliche Situationen** gedacht.
Z. B. I/O-Probleme
- Exceptions werden auch für **Preconditions** verwendet.
- Verwende Exception, wenn es **keine alternative Handlung** gibt.
D. h., es lässt sich nichts weiter machen, außer den Fehler auszugeben.
- Verwende Exceptions **nicht**, um den **Kontrollfluss zu steuern**.
D. h., wenn sich etwas leicht mit if-else prüfen lässt, dann nutze if-else.

```
// do not do this:  
Student student = new Student("Sara");  
try {  
    return student.name(); // student can be null  
} catch (NullPointerException e) {  
    return "";  
}
```

```
// do this instead:  
if(student == null) {  
    return "";  
}  
return student.name();
```

```
// or even better use an Optional<Student>:  
return student.map(Student::name).orElse("");
```

Wann Optional<T> verwenden?

- Optional<T> sollte nur **für Rückgabewerte von Methoden** verwendet werden.
- **Ausnahme:** Wenn eine **Methode ein Iterable (Collection) zurückgibt**, ist es in der Regel besser, entweder **die Ergebnisliste** zurückzugeben oder **eine leere Liste** (z. B. bei Suchanfragen).
 - Ausnahme von der Ausnahme:
 - Eine Ergebnisliste darf nicht leer sein (z. B., um Division durch 0 zu vermeiden).
 - Es soll eine explizite Fehlermeldung an den User kommuniziert werden.
- Verwende Optional<T> **nicht für Parameter!**
Verwende stattdessen Method-Overloading oder Varargs.

Wiederholung: Lambda



Ein Lambda ist eine **anyonyme Methode**, die **lazy evaluiert** wird, d. h. sie wird **erst implementiert** und **zu einem späteren Zeitpunkt ausgeführt** (evaluiert).

```
@FunctionalInterface
public interface MoneyPrinter {
    void amount(int value);
}
```

```
public static void main(String[] args) {
```

```
    MoneyPrinter moneyPrinter = (int value) -> System.out.print("€" + value);
```

```
    moneyPrinter.amount(42);
```

```
}
```

Evaluation

↑
Typ kann weggelassen werden,
da dieser von Java inferierbar ist.

Implementation

Ausgabe:
€42

NEXT LEVEL

Higher-Order Functions

$f(g(x))$

Das Konzept der **Higher-Order Function** stammt aus der funktionalen Programmierung.

Es handelt sich um eine Funktion (Methode), die eine oder mehrere der folgenden Eigenschaften hat:

1. Nimmt eine oder mehrere **Funktionen als Argument(e)**.
Beispiele: map, filter, reduce, sorted, forEach, orElseGet, orElseThrow
2. Gibt eine **Funktion als Ergebnis** zurück.
Beispiele: Collectors.*, Comparator.*, Stream.generate

NEXT LEVEL

Higher-Order Functions

$f(g(x))$

Beispiel:

`Comparator.comparing(Person::getAge)`

Die Methode `comparing` ist eine HOF. Ihr gebt ihr eine Funktion (hier die Methodenreferenz `Person::getAge`) und was bekommt ihr zurück?

Nicht das Alter, sondern eine *neue Funktion*, nämlich einen fertigen `Comparator`, der Personen nach Alter sortieren kann."

Oder:

`Collectors.toList()`

Das ist eine Methode, die eine Funktion zurückgibt, die der `collect`-Methode sagt, wie sie Elemente in einer Liste sammeln soll."

Pure Lambda



Lambdas sollen "pure" sein, d. h. **keine Seiteneffekte bzw. keinen Zustand** haben, damit sich Higher-Order Functions **vorhersagbar (deterministisch)** und **parallelisierbar** ausführen lassen:

1. Pure Lambda nimmt keine nach außen sichtbaren Veränderungen vor.
2. Pure Lambda hängt nicht von etwas ab, das sich von außen verändern kann.

Beispiel: `add(a, b)` ist pure. Das Ergebnis hängt nur von `a` und `b` ab."

Beispiel: `add(a)` das intern `a + globalVariable_b` rechnet, ist *nicht* pure, weil `globalVariable_b` sich ändern könnte.

Callbacks



Ein **Callback** ist eine bestimmte Art von **Higher-Order Function**.

Hierbei wird **eine Funktion (Lambda) als Argument** an eine **andere Funktion übergeben**, welche die übergebene Funktion später aufruft (Lazy Evaluation).

Callbacks ermöglichen es, auf **Ereignisse zu reagieren**, sobald diese eintreten.

Beispiele:

- Button in GUI-Frameworks, um Ereignisse wie `onClick(lambda)` zu reagieren.
- Abschluss eines Netzwerktransfers `onDownloadCompleted(lambda)`.
- Methoden mit `@BeforeEach` und `@AfterEach` in Unit-Tests sind eine spezielle Form von Callbacks (mittels Annotationen realisiert statt Lambdas).

Begriffe: Callback, Event, Hook

Callback: Allgemeiner Begriff für eine Funktion, die als Parameter übergeben und zu einem späteren Zeitpunkt aufgerufen wird.



Event: Bezeichnet das Reagieren auf ein Ereignis, sobald dieses eintritt, z. B. eine Benutzeraktion oder ein Systemereignis. Der Programmablauf kann durch ein Event unterbrochen oder parallel (asynchron) verarbeitet werden. Events lassen sich mittels Callbacks realisieren.



Hook: [Hooks](#) sind spezielle Callbacks, die oft von Frameworks bereitgestellt werden. Durch Hooks erhält man die Möglichkeit, eigenen Code in die fremden Funktionen eines Frameworks "einzuschleusen".



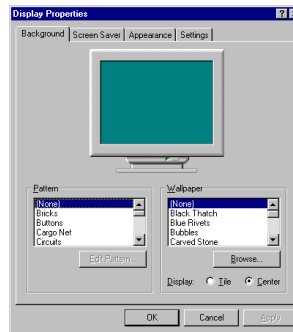
Callback: Beispiel 1

```
class DownloadManager {  
  
    void downloadFile(String url, Consumer<String> callback) {  
        System.out.println("Datei wird heruntergeladen...");  
        // ... downloading file from url ...  
        String message = "Download abgeschlossen!";  
        callback.accept(message); // evaluate callback  
    }  
}  
  
class DownloadSimulator {  
  
    public static void main(String[] args) {  
        DownloadManager downloader = new DownloadManager();  
        downloader.downloadFile(  
            "https://de.wikipedia.org/wiki/R%C3%BCckrufffunktion#/media/Datei:ProgramExecCallbackSimple-de.png",  
            message -> System.out.println(message) // provide a callback method that is called later.  
        );  
    }  
}
```

Ausgabe:

```
Datei wird heruntergeladen...  
Download abgeschlossen!
```

Callback: Beispiel 2 (Swing)



```
class DisplayPropertiesView extends JView {  
  
    private final Properties properties = new Properties();  
  
    private final JButton okButton = new JButton();  
    private final JButton cancelButton = new JButton();  
  
    DisplayPropertiesView() {  
        initializeWidgets();  
    }  
  
    private void initializeWidgets() {  
        cancelButton.onClicked(() -> properties.discard());  
  
        okButton.onClicked(() -> {  
            properties.save();  
            Popup.show("Properties saved");  
        });  
    }  
}
```

```
class JButton { // this class is part of a GUI framework  
  
    private Runnable action; // stored action  
  
    void onClicked(Runnable action) {  
        this.action = action; // store action  
    }  
  
    // click is called by GUI framework when user clicks this button  
    void click() {  
        action.run(); // evaluate stored action  
    }  
}
```

```
@FunctionalInterface  
public interface Runnable {  
  
    void run();  
}
```


Callback: Unit Test

Callbacks müssen oftmals **anders getestet** werden, da die eigentliche Methode, die getestet werden soll, oft **void zurückgibt** (muss aber nicht).

Im Testfall wird zunächst eine eigene Implementation eines Callbacks mitsamt der **assert-Methoden** übergeben (assert).

Erst anschließend wird die **zu testende Methode aufgerufen** (act).

Act und **Assert** werden in diesem Fall **in anderer Reihenfolge** implementiert. 🤪

```
@Test
void bakePublishesOnCompleteCallback() throws InterruptedException {
    // Arrange
    var waffleMaker = new NuclearPoweredWaffleMaker();

    // Assert (im Callback)
    waffleMaker.subscribeOnComplete(message -> { // subscribe a custom callback for this testcase
        assertEquals("Waffle ready!", message); // assert is within the test callback
    });

    // Act
    waffleMaker.bake(); // method to be tested: must be called after subscribe!
}
```



Callbacks: Wozu?



In dem vorherigem Beispiel ☢️ NuclearPoweredWaffleMaker könnte man die Callbacks doch einfach direkt in der bake-Methode implementieren? Wozu Callbacks? #WTF

Separation of Concerns: Callbacks ermöglichen es, den Backvorgang von den nachgelagerten Aktionen zu trennen. Die bake-Methode kümmert sich nur um das Backen und nicht darum, was danach passiert. Dies führt zu saubererem und modularerem Code.

Wiederverwendbarkeit und Flexibilität: Durch die Verwendung von Callbacks lassen sich beliebige Aktionen nach dem Backen ausführen, ohne die bake-Methode ändern zu müssen. Dies macht den Code wiederverwendbarer und flexibler.

Erweiterbarkeit: Neue Funktionen können hinzugefügt werden, indem neue Callbacks registriert werden, anstatt die bestehende Logik zu ändern und testen zu müssen.

Event-gesteuerte Programmierung: Callbacks sind ein wesentlicher Bestandteil der asynchronen Programmierung und ermöglichen es, auf Ereignisse zu reagieren, sobald sie eintreten, ohne den Hauptfluss des Programms zu blockieren.

List<String>

<T extends Number & Comparable<T>>

Generic Methods und Types



Object



Generics

Generic Methods

Generische Methoden sind **Methoden**, die **selbst Typparameter definieren** – unabhängig davon, ob die Klasse, in der sie sich befinden, generisch ist oder nicht.

Sie ermöglichen es, Typen flexibel zu verwenden, ohne sich auf einen bestimmten Datentyp festzulegen. Diese Typparameter werden **direkt vor dem Rückgabetyp** der Methode angegeben.

Beispiel: SimpleBag<E>



Mithilfe von Generics ist es möglich, Klassen zu erstellen, die mit einem bestimmten Datentyp arbeiten. Der Bezeichner zwischen den <> kann beliebig gewählt werden.

```
class SimpleBag<E> {  
    private final E element;  
  
    public SimpleBag(E element) {  
        this.element = element;  
    }  
  
    public E getElement() {  
        return this.element;  
    }  
  
    public static void main(String[] args) {  
        SimpleBag<Integer> bagWithNumber = new SimpleBag<>(42);  
        System.out.println(bagWithNumber.getElement());  
  
        SimpleBag<String> bagWithName = new SimpleBag<>("Moin");  
        System.out.println(bagWithName.getElement());  
    }  
}
```

Konvention für Platzhalter:

E - Element (oft in Collections)

K - Key

N - Number

T - Type

V - Value

S, U, etc. für weitere Typen

Ausgabe:

42

Moin

Erinnerung: Einschränkungen für Generics

Primitive Typen:

Generische Typen können **keine** primitiven Datentypen sein (z. B. <int>, <char>). Stattdessen müssen Wrapper-Klassen (z. B. <Integer>, <Character>) verwendet werden.

Nachteil: `null`-Objekte sind möglich. :-/

Typinformationen zur Laufzeit:

Wegen Type Erasure stehen Typparameter zur Laufzeit nicht zur Verfügung. Man kann daher nicht direkt auf den Typparameter zugreifen oder instanziiieren.

Beispiel: `new T()` und `obj instanceof T` funktionieren nicht (es gibt Workarounds).

Generics: Invarianz

Definition: Invarianz bedeutet, dass `GenericType<SubType>` kein Subtyp oder Supertyp von `GenericType<SuperType>` ist – selbst wenn `SubType` ein Untertyp von `SuperType` ist.

→ Ein generischer Typ ist **nur exakt mit dem deklarierten Typ** kompatibel – **nicht mit Unter- oder Obertypen.**

Beispiel:

`ArrayList<Object> numbers = new ArrayList<Integer>();` *// does not compile!*

Generics: Invarianz Beispiel 1

String ist ein **Subtyp** von **Object** (String extends Object).

```
String greetAsString = "Hallo";  
Object greetAsObject = greetAsString;  
System.out.println(greetAsObject);
```

ABER:

List<String> ist **kein Subtyp** von **List<Object>**! *// Invarianz*

Der Grund ist, Typensicherheit zu gewährleisten:

```
List<Integer> listOfIntegers = new ArrayList<>(List.of(1, 2));  
List<Object> listOfObjects = listOfIntegers; // nicht kompilierbar  
listOfObjects.add("Hallo"); // würde einen String in eine List<Integer> hinzufügen.
```


Wildcards vs <T>

Ein **Typ-Parameter (T)** definiert eine feste Variable für einen Typen, den du innerhalb der Methode referenzieren musst, um etwa Konsistenz zwischen Parametern und Rückgabewerten zu garantieren.

Eine **Wildcard (?)** steht hingegen für einen unbekannten Typen und wird verwendet, wenn der exakte Inhalt für die Operation irrelevant ist (meistens nur zum Lesen).

Kurz gesagt: Nutze $\mathbb{T}$, wenn du den Typen "anfassen" oder zurückgeben musst, und $?$, wenn dir bloße Flexibilität beim Empfangen von Daten reicht.

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) **T mit extends:** wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}
```

```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) **T mit extends:** wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}
```

```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```

```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) T mit extends: wenn der Parameter gelesen und/oder geschrieben wird.

- ```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
 numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
 for (Object number : numbers) // Jedes Element ist von Typ Object.
 System.out.println(number);
}
```
- ```
(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}
```
- ```
(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
 numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
 for (Number number : numbers) // Jedes Element ist von Typ Number.
 System.out.println(number);
}
```
- ```
(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) T mit extends: wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}

(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}

(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}

(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Wildcards vs <T>



(1) **Wildcards mit super:** wenn der Parameter modifiziert wird (z. B. durch add-/remove-Methoden).

(2) **Wildcards mit extends:** wenn der Parameter nur gelesen bzw. iteriert wird.

(3 und 4) **T mit extends:** wenn der Parameter gelesen und/oder geschrieben wird.

```
(1) void addIntegers(List<? super Integer> numbers) { // Wildcards mit super. Akzeptierte Typen sind: List<Integer>, List<Number>, List<Object>
    numbers.add(1); // Es dürfen Integer und Untertypen von Integer (wenn Integer Untertypen hätte) hinzugefügt werden.
    for (Object number : numbers) // Jedes Element ist von Typ Object.
        System.out.println(number);
}

(2) void print(List<? extends Number> numbers) { // Wildcards mit extends. Akzeptierte Typen sind z. B. List<Number>, List<Integer>, List<Double>, usw.
    // numbers.add nicht erlaubt (wegen extends)
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}

(3) <T extends Number> void print(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    for (Number number : numbers) // Jedes Element ist von Typ Number.
        System.out.println(number);
}

(4) <T extends Number> List<T> reversed(List<T> numbers) { // Hier ist nur extends erlaubt. <T super Number> ist nicht möglich.
    numbers.add((T) (Number) 4); // Schreiben erlaubt mit Cast (hier wäre Cast zu Number oder Object erlaubt).
    return numbers.reversed(); // Rückgabewert muss denselben Typ wie numbers haben. Mit Wildcards wäre das nicht modellierbar.
}
```

- Wenn das Generic nur einmal vorkommt, wird i. d. R. die Wildcards-Variante (2) statt (3) vorgezogen.
- Wenn das Generic mehrmals vorkommt und man Typgleichheit erzwingen will, ist die T-Variante notwendig (4).
- Beschränkung mit super ist nur für Wildcards erlaubt (1), nicht aber mit T (3 und 4).

Komplexe Typbeziehungen: Übersicht

Konzept	Beispiel	Erlaubte Zuweisungen
Invarianz	<code>List<Object> list = new ArrayList<Object>();</code>	<code>list</code> akzeptiert ausschließlich <code>List<Object></code>
Kovarianz	<code>List<? extends Number> list = new ArrayList<Integer>();</code> Nur Lesen von <code>list</code> erlaubt. Elemente müssen als <code>Number</code> oder eine Oberklasse von <code>Number</code> gelesen werden.	<code>list</code> akzeptiert <code>List<Number></code> und alle Listen mit Untertypen von <code>Number</code> .
Kontravarianz	<code>List<? super Number> list = new ArrayList<Serializable>();</code> Hinzufügen von <code>Number</code> oder Unterklassen von <code>Number</code> in <code>list</code> erlaubt. Lesen von <code>list</code> erlaubt, aber Elemente müssen als <code>Object</code> gelesen werden.	<code>list</code> akzeptiert <code>List<Number></code> und alle Listen mit Obertypen von <code>Number</code> .
Type-Bounding	<code>static <T extends Number> void print(List<T> list) { ... }</code> <code>static void main(String[] args) {</code> <code>print(new ArrayList<Integer>());</code> <code>}</code> Hinzufügen mit Cast zu <code>(T)(Number)</code> in <code>list</code> erlaubt. Lesen von <code>list</code> erlaubt, aber Elemente müssen als <code>T</code> , <code>Number</code> oder eine Oberklasse von <code>Number</code> gelesen werden.	<code>print</code> akzeptiert <code>List<Number></code> und alle Listen mit Untertypen von <code>Number</code> .



PECS (Producer Extends, Consumer Super)

Producer Extends (Upper Bound / Kovarianz)

Verwende ? extends Type, wenn Daten aus einer Struktur **gelesen** werden.
Die Liste numbers "produziert" hier die Elemente.

```
// Accept List<Number> and all subtypes of Number, e.g., List<Integer>, List<Double> ...  
void readNumbers(List<? extends Number> numbers) {  
    for (Number number : numbers) {  
        System.out.println(number);  
    }  
}
```

Consumer Super (Lower Bound / Kontravarianz)

Verwende ? super Type, wenn Daten in eine Struktur **geschrieben** werden.
Die Liste numbers "konsumiert" hier die Elemente.

```
// Accept List<Integer> and any super types of Integer, e.g., List<Number> ...  
void addIntegers(List<? super Integer> numbers) {  
    numbers.add(1);  
    numbers.add(2);  
    numbers.add(3.3); // does not compile, must be Integer  
}
```


Pattern und Matcher

Die Methoden `String::replaceAll`, `String::split` und `String::matches` verwenden intern die Klassen **Pattern** und **Matcher** für die Auswertung der regulären Ausdrücke.

Ein regulärer Ausdruck wird zunächst mit der Klasse **Pattern** (Muster) kompiliert (optimiert). Das Kompilieren sollte aus Performance-Gründen nur einmalig erfolgen (nicht in Schleifen).

Die Klasse **Pattern** hat die Methode **matcher(String)**, die einen Eingabe-String erhält und einen **Matcher** zurückgibt.

```
public static void main(String[] args) {  
    Pattern pattern = Pattern.compile("A(\\w)*d", Pattern.CASE_INSENSITIVE);  
    String input = "Android Ad Abenteuer bald Abend";  
    Matcher matcher = pattern.matcher(input);  
    while (matcher.find()) { // prüft, ob es einen weiteren Match gibt (true/false).  
        System.out.println(matcher.group()); // group() gibt den nächsten Match als String zurück.  
    }  
}
```

Ausgabe:
Android
Ad
ald
Abend

```
System.out.println(matcher.group()); // IllegalStateException: No match found → find() gibt false zurück.  
}
```

Pattern und Matcher: Groups

Mit runden Klammern lassen sich Gruppen im regulären Ausdruck definieren, die sich durch Indizes über `Matcher::group` abrufen lassen.

```
public static void main(String[] args) {  
    Pattern pattern = Pattern.compile("(\\d{3})-(\\d{2})-(\\d{4})");  
    String input = "123-45-6789";  
    Matcher matcher = pattern.matcher(input);  
    if (matcher.matches()) { // prüft, ob der input als Ganzes matched (true/false).  
        System.out.println("Full match: " + matcher.group(0));  
        System.out.println("Group 1: " + matcher.group(1));  
        System.out.println("Group 2: " + matcher.group(2));  
        System.out.println("Group 3: " + matcher.group(3));  
    }  
}
```

Ausgabe:

```
Full match: 123-45-6789  
Group 1: 123  
Group 2: 45  
Group 3: 6789
```

```
Matcher matcher2 = pattern.matcher("abc 123-45-6789 xyz");  
matcher2.matches(); // false → der ganze Text passt nicht aufs Muster (Pattern).  
matcher2.find();    // true → weil ein Teilstring ("123-45-6789") passt.  
}
```

Non-Capturing Groups

Wenn runde Klammern () in einem regulären Ausdruck verwendet werden, erzeugt das eine **Capturing Group**, die sich mittels `matcher.group(n)` abfragen lässt.

Wenn die Klammerung **nur für die Gruppierung der Zeichen, aber nicht zur Speicherung** verwenden werden soll, wird eine sogenannte **non-capturing group** benötigt, die **mit ?: beginnt**.

```
public static void main(String[] args) {  
    Pattern pattern = Pattern.compile("(?:\\d+)-(abc)+");  
    String input = "123-abcabc";  
    Matcher matcher = pattern.matcher(input);  
    if (matcher.matches()) {  
        System.out.println(matcher.group(1)); // gibt abc aus. Ließe man das ?: weg, würde man stattdessen 123 erhalten.  
    }  
}
```

- `(?:\\d+)` wird nur für das Regex + gruppiert, ist aber **nicht** mittels `group(1)` abrufbar.
- `(abc)+` ist eine Capturing Group und kann mittels `group(1)` abgerufen werden.

Achtung: Bei einer Wiederholung (+, *, {n,m}) gibt `group(n)` **den letzten erfolgreichen Match** der Gruppe zurück. Will man stattdessen den gesamten Teil "abcabc" erhalten, muss man `"(?:\\d+)-((?:abc)+)"` als Regex verwenden.

UrlParameterParser

Gegeben ist folgender **Unit Test** für die Klasse **UrlParameterParser**, der **aus einer URL** alle **Parameter als Map extrahiert**.
Implementiere die zugehörige Klasse **UrlParameterParser**!

Hinweise:

Parameter **starten nach** dem **ersten ?** in der URL und werden **anschließend** mit **&** **getrennt**.

Parameter haben immer einen **Key** und einen **Value**. Key und Value werden mit einem **=** **getrennt**.

Beispiel: `https://haw-hamburg.de?parameter1=value1¶meter2=value2¶meter3=value3`

Test zeigen

```
public class UrlParameterParser {  
  
    public static Map<String, String> parse(String url) {  
        Map<String, String> params = new HashMap<>();  
  
        // TODO  
  
        return params;  
    }  
}
```

Warum Hashmap?

```
public static Map<String, String> parse(String url) {  
    Map<String, String> params = new HashMap<>();
```

```
// 1. Alles hinter dem ? finden
```

```
Pattern queryPattern = Pattern.compile("\\?(.*)$");
```



```
public static Map<String, String> parse(String url) {  
    Map<String, String> params = new HashMap<>();  
  
    // 1. Alles hinter dem ? finden  
    Pattern queryPattern = Pattern.compile("\\?(.*)$");  
    Matcher queryMatcher = queryPattern.matcher(url);
```

```
public static Map<String, String> parse(String url) {  
    Map<String, String> params = new HashMap<>();  
  
    // 1. Alles hinter dem ? finden  
    Pattern queryPattern = Pattern.compile("\\?(.*)$");  
    Matcher queryMatcher = queryPattern.matcher(url);  
  
    if (queryMatcher.find()) {  
        String queryString = queryMatcher.group(1);  
        // Jetzt haben wir z.B. "param1=value1&param2=value2"  
  
        // TODO: Parameter zerlegen  
    }  
}
```

```
public static Map<String, String> parse(String url) {  
    Map<String, String> params = new HashMap<>();  
  
    // 1. Alles hinter dem ? finden  
    Pattern queryPattern = Pattern.compile("\\?(.*)$");  
    Matcher queryMatcher = queryPattern.matcher(url);  
  
    if (queryMatcher.find()) {  
        String queryString = queryMatcher.group(1);  
  
        // 2. Key-Value Paare finden  
        // Gruppe 1: Key (mindestens 1 Zeichen, kein & oder =)  
        // Gruppe 2: Value (beliebig viele Zeichen, kein & oder =)  
        Pattern paramPattern = Pattern.compile("([^&=]+)=([^&=]*)");  
        Matcher paramMatcher = paramPattern.matcher(queryString);  
  
        // TODO: Durchlaufen  
    }  
  
    return params;  
}
```

```
if (queryMatcher.find()) {  
    String queryString = queryMatcher.group(1);  
  
    // 2. Key-Value Paare finden  
    // Gruppe 1: Key (mindestens 1 Zeichen, kein & oder =)  
    // Gruppe 2: Value (beliebig viele Zeichen, kein & oder =)  
    Pattern paramPattern = Pattern.compile("([^&=]+)=([^&=]*)");  
    Matcher paramMatcher = paramPattern.matcher(queryString);  
  
    while (paramMatcher.find()) {  
        String key = paramMatcher.group(1);  
        String value = paramMatcher.group(2);  
        params.put(key, value);  
    }  
}
```

Rekursion

Rekursion (lat. “zurücklaufen”) tritt auf, wenn eine Methode sich **selbst direkt** oder **indirekt aufruft**.

Jede rekursive Methode muss eine **erreichbare Abbruchbedingung** haben, um eine Endlos-Rekursion zu vermeiden (ansonsten Stackoverflow).

Rekursion wird oft verwendet, um Probleme zu lösen, die sich in kleinere, **ähnliche Teilprobleme zerlegen** ("Teile und Herrsche") lassen.

Rekursion: Beispiel 1/2

```
class MathUtil {
```

```
    static int faculty(int n) {  
        // Abort condition  
        if (n == 0) {  
            return 1;  
        }  
        // Recursive loop  
        return n * faculty(n - 1);  
    }
```

```
    public static void main(String[] args) {  
        int n = 3;  
        int result = faculty(n);  
        System.out.printf("%d! = %d", n, result); // Output: 3! = 6  
    }  
}
```



"Die Fakultät ist in der Mathematik diejenige Funktion, die jeder natürlichen Zahl das Produkt aller positiven natürlichen Zahlen zuordnet, die diese Zahl nicht übertreffen. Sie wird durch ein dem Funktionsargument nachgestelltes Ausrufezeichen („!“) abgekürzt." – Wikipedia

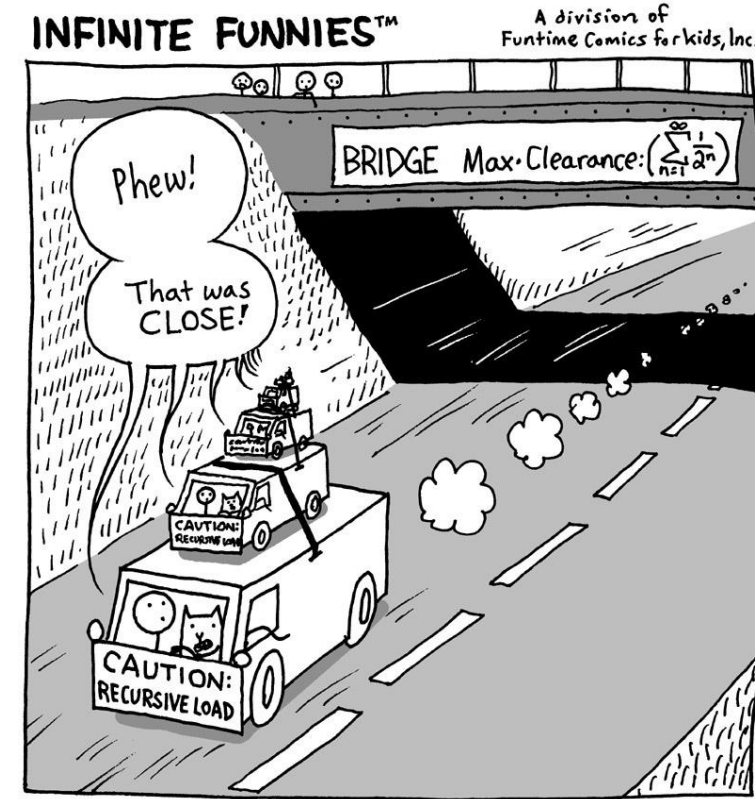
Rekursion: Beispiel 2/2

```
class MathUtil {
```

```
    static int faculty(int n) {  
        // Abort condition  
        if (n == 0) {  
            return 1;  
        }  
        // Recursive loop  
        return n * faculty(n - 1); // n * faculty(n); führt zur Endlosrekursion.  
    }
```

Damit die Abbruchbedingung in diesem Beispiel erreichbar ist, darf der Parameter **nicht konstant** sein!

```
    public static void main(String[] args) {  
        int n = 3;  
        int result = faculty(n);  
        System.out.printf("%d! = %d", n, result); // Output: 3! = 6  
    }  
}
```



"Die Fakultät ist in der Mathematik diejenige Funktion, die jeder natürlichen Zahl das Produkt aller positiven natürlichen Zahlen zuordnet, die diese Zahl nicht übertreffen. Sie wird durch ein dem Funktionsargument nachgestelltes Ausrufezeichen („!“) abgekürzt." – Wikipedia

Wie gehe ich an Rekursionsaufgaben ran?

- Schreibe eine Java Funktion, die rekursiv zählt, wie oft ein bestimmtes Zeichen in einem String vorkommt.


```
@Override  
public int countRecursively(String text, char searchChar) {  
    //TODO  
    //return 0;  
    return countHelper(text, searchChar, 0);  
}
```

```
// Helper method | Hilfsmethode  
// Diese Methode erledigt die eigentliche Arbeit mit dem Index  
private int countHelper(String text, char searchChar, int index) {  
    // TODO  
    if (index == text.length()) {  
        return 0;  
    }  
}
```

// Helper method | Hilfsmethode

// Diese Methode erledigt die eigentliche Arbeit mit dem Index

```
private int countHelper(String text, char searchChar, int index) {
```

```
    // TODO
```

```
    if (index == text.length()) {
```

```
        return 0;
```

```
    }
```

```
    int match = 0;
```

```
    if (text.charAt(index) == searchChar) {
```

```
        match = 1;
```

```
    }
```

```
}
```

```
private int countHelper(String text, char searchChar, int index) {  
    // TODO  
    if (index == text.length()) {  
        return 0;  
    }  
  
    int match = 0;  
    if (text.charAt(index) == searchChar) {  
        match = 1;  
    }  
  
    int restResult = countHelper(text, searchChar, index + 1);  
}
```

```
private int countHelper(String text, char searchChar, int index) {  
    // TODO  
    if (index == text.length()) {  
        return 0;  
    }  
  
    int match = 0;  
    if (text.charAt(index) == searchChar) {  
        match = 1;  
    }  
  
    int restResult = countHelper(text, searchChar, index + 1);  
  
    return match + restResult;  
}
```

```
private int countHelper(String text, char searchChar, int index) {  
  
    if (index == text.length()) {  
        return 0;  
    }  
  
    char charInText = Character.toLowerCase(text.charAt(index));  
    char charToSearch = Character.toLowerCase(searchChar);  
  
    int match = 0;  
    if (charInText == charToSearch) {  
        match = 1;  
    }  
  
    return match + countHelper(text, searchChar, index + 1);  
}
```

CompletableFuture<T>



CompletableFuture<T> (auch **Promise** oder **await/async**) ist (neben FutureTask) eine Java-Implementation von **Future<V>**.

Ein CompletableFuture wird ebenfalls **asynchron** ausgeführt, d. h., das Java-Programm läuft weiter, während das CompletableFuture ausgeführt wird.

Die **get()**-Methode **blockiert** den aktuellen Thread, bis das Ergebnis verfügbar ist. Sie wirft eine **Checked Exception**.

CompletableFuture<T> akzeptiert ein **Supplier<T>** und bietet neben get() zusätzlich die Methode **join()**, die wie get() auch **blockiert und ein Ergebnis zurückgibt**, aber nur **Unchecked Exceptions** wirft. 🤪

Darüber hinaus bietet CompletableFuture **Method-Chaining** (aka Fluent-API), um mehrere Aufgaben miteinander **verketteten** oder **kombinieren** zu können (z. B. supplyAsync, thenApply, exceptionally, thenAccept, thenRun, join).

Threads und CompletableFutures

- **Grundkonzept**

- **Der Thread** ist der "Arbeiter". Er ist die Ressource des Betriebssystems, die den Code tatsächlich ausführt. Threads sind teuer in der Erstellung und Verwaltung.
- **Das CompletableFuture** ist ein "Auftrag" oder "Versprechen" (Promise) auf ein zukünftiges Ergebnis. Es ist eine Abstraktion, die sagt: *"Hier ist eine Aufgabe, erledige sie irgendwann und gib mir Bescheid (oder das Ergebnis), wenn du fertig bist."*

supplyAsync

- Wenn du **supplyAsync** aufrufst, passiert "unter der Haube" Folgendes:
- Das `CompletableFuture` sucht sich einen freien Thread (standardmäßig aus dem globalen `ForkJoinPool.commonPool()`).
- Dieser **Thread** führt die Methode im Hintergrund aus.
- Der **Haupt-Thread** wartet nicht. Er holt sich sofort das Future-Objekt (den "Abholschein") und geht zum nächsten Service weiter. Dadurch laufen alle Abfragen **parallel**.

return future.join();

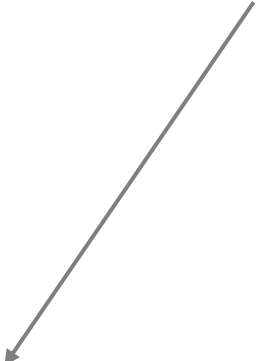
- Die Methode join() blockiert den Thread, der sie aufruft.
- Er wartet, bis der Hintergrund-Thread mit der Arbeit fertig ist und das Ergebnis in das CompletableFuture gelegt hat.
- Da du die Futures *vorher* in einer Schleife gestartet hast, laufen sie bereits parallel, während du hier nur noch die Ergebnisse einsammelst.

CompletableFuture<T>: Beispiel

```
@FunctionalInterface
public interface Supplier<T> {

    T get();

}
```



```
public static void main(String[] args) {
    CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> 42);
    // Awesome code, der ausgeführt wird, während das Future asynchron berechnet wird...
    System.out.println(future.join()); // join() blockiert, bis future fertig ist und gibt dann das 42 als Rückgabewert zurück.
}
```

Hinweis: Im Gegensatz zu Future<V> verwendet CompletableFuture<T> das Interface **Supplier<T>** statt Callable<V>. Beide Interfaces sind fast gleich, Supplier<T> wirft allerdings keine Checked Exception.

CompletableFuture<T>: Beispiel

```
@FunctionalInterface
public interface Supplier<T> {

    T get();

}
```

```
public static void main(String[] args) {
    CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> 42);
    // Awesome code, der ausgeführt wird, während das Future asynchron berechnet wird...
    System.out.println(future.join()); // join() blockiert, bis future fertig ist und gibt dann das 42 als Rückgabewert zurück.
}
```

Hinweis: Im Gegensatz zu Future<V> verwendet CompletableFuture<T> das Interface **Supplier<T>** statt Callable<V>. Beide Interfaces sind fast gleich, Supplier<T> wirft allerdings keine Checked Exception.

Future<V> vs CompletableFuture<T>

Erzeugung:

```
try(ExecutorService executor = Executors.newFixedThreadPool(4)) {  
    Future<V> future = executor.submit(Callable<V> callable);  
    // ...  
}
```

```
CompletableFuture<T> completableFuture = CompletableFuture.supplyAsync(Supplier<T> supplier);
```

Verwendung:

```
V result = future.get(); // blocks current thread, throws Checked Exception
```

```
T result = completableFuture.join(); // blocks current thread, throws Unchecked Exception  
T result = completableFuture.get(); // blocks current thread, throws Checked Exception
```

CompletableFuture<T> implementiert
(ist auch) ein Future<T>.

CompletableFuture<T>: Beispiel Chaining 1/2

```
public static void main(String[] args) {  
    CompletableFuture<String> future = CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            System.out.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde  
            System.err.println(ex.getMessage());  
            return null;  
        })  
  
    System.out.println("Join...");  
    var result = future.join(); // Blockiert, bis Ergebnis "Hello World!" verfügbar ist. Im Gegensatz zu get(), wirft join() nur Unchecked Exceptions.  
    System.out.println(result);  
    System.out.println("End.");  
}
```

Ausgabe:

```
Task 1  
Join...  
Task 2  
Task 3  
Hello World!  
End.
```

CompletableFuture<T>: Beispiel Chaining 1/2

```
public static void main(String[] args) {  
    CompletableFuture<String> future = CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            System.out.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde  
            System.err.println(ex.getMessage());  
            return null;  
        })  
  
    System.out.println("Join...");  
    var result = future.join(); // Blockiert, bis Ergebnis "Hello World!" verfügbar ist. Im Gegensatz zu get(), wirft join() nur Unchecked Exceptions.  
    System.out.println(result);  
    System.out.println("End.");  
}
```

Ausgabe:

```
Task 1  
Join...  
Task 2  
Task 3  
Hello World!  
End.
```

CompletableFuture<T>: Beispiel Chaining 1/2

```
public static void main(String[] args) {  
    CompletableFuture<String> future = CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            System.out.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde  
            System.err.println(ex.getMessage());  
            return null;  
        })  
  
    System.out.println("Join...");  
    var result = future.join(); // Blockiert, bis Ergebnis "Hello World!" verfügbar ist. Im Gegensatz zu get(), wirft join() nur Unchecked Exceptions.  
    System.out.println(result);  
    System.out.println("End.");  
}
```

Ausgabe:

```
Task 1  
Join...  
Task 2  
Task 3  
Hello World!  
End.
```


CompletableFuture<T>: Beispiel Chaining 1/2

```
public static void main(String[] args) {  
    CompletableFuture<String> future = CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            System.out.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde  
            System.err.println(ex.getMessage());  
            return null;  
        })  
    ;  
  
    System.out.println("Join...");  
    var result = future.join(); // Blockiert, bis Ergebnis "Hello World!" verfügbar ist. Im Gegensatz zu get(), wirft join() nur Unchecked Exceptions.  
    System.out.println(result);  
    System.out.println("End.");  
}
```

Ausgabe:

```
Task 1  
Join...  
Task 2  
Task 3  
Hello World!  
End.
```

CompletableFuture<T>: Beispiel Chaining 2/2

```
public static void main(String[] args) {  
    CompletableFuture  
        .supplyAsync(() -> { // Supplier<U>. Startet asynchrone Aufgabe.  
            System.out.println("Task 1");  
            return "Hello";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 2");  
            return previousResult + " World";  
        })  
        .thenApply(previousResult -> { // Function<T, U>. Verkettete Aufgabe.  
            System.out.println("Task 3");  
            return previousResult + "!";  
        })  
        .exceptionally(ex -> { // Wird ausgeführt, falls eine Exception zuvor geworfen wurde.  
            System.err.println(ex.getMessage());  
            return null;  
        })  
        .thenAccept(result -> System.out.println(result)) // Verarbeitet das Endergebnis.  
        .thenRun(() -> System.out.println("End."))  
        .join(); // Blockiert den main-Thread, ansonsten kann es sein, dass das CompletableFuture abgebrochen wird (shutdown).  
}
```

Ausgabe:

```
Task 1  
Task 2  
Task 3  
Hello World!  
End.
```

In dieser Variante wird alles in der CompletableFuture-Chain durchgeführt.

Recap

Thread: Java-Klasse, mit der Code nebenläufig ausgeführt werden kann (verwendet Runnable-Interface).

synchronized: Java-Schlüsselwort um thread-safe zu programmieren. Für feingranulare Optimierung.

Collections.synchronized*: Macht eine beliebige Collection-Klasse thread-safe.

Concurrent: Package mit vielen nützlichen thread-safe Klassen.

ExecutorService: Verteilt Aufgaben (Tasks) an einen Thread-Pool. Threads werden dabei wiederverwendet.

Runnable: Functional Interface, ohne Rückgabewert, das als Thread ausgeführt werden kann.

Callable: Functional Interface ähnlich wie Runnable, das aber ein Ergebnis zurückgibt (wie Supplier<T>). Kann von ExecutorService als Thread ausgeführt werden und gibt ein Future<V> zurück.

Supplier wirft aber keine Exception.



Future<V> (aka Promise bzw. await/async): Interface. Berechnet Ergebnis asynchron. Methode get() blockiert und liefert Ergebnis, wirft aber Checked Exception. Beispiel: Future<V> executor.submit(Callable<V> task);

CompletableFuture<T>: Implementiert Future<V> mit Fluent-API. Verwendet statt Callable<V> das Supplier<T>-Interface. Methoden get() und join() liefern das Ergebnis (blockierend), join() wirft aber nur eine Unchecked Exception.

HttpClient: Internet-Ressourcen anhand einer URI abrufen. Jede Anfrage (request) gibt ein CompletableFuture<T> (response) mit dem Ergebnis zurück.

Schaut euch genauer an:

- Callbacks
- Collections
- Concurrency
- Generics
- Parser (Regex)
- Rekursion
- Lambda und Streams
- Unit Tests
- Exceptions